Actor Graphs: Triple-Identity Accountable Mediation and Coinductive Disclosure

Identity-Faithful Representations, Transaction-Sourced Accounts, and a Presheaf Accountability Skeleton

Alexander Vityaz
Corezoid Inc., Dnipro, Ukraine
alexander.vityaz@gmail.com
corezoid.com · simulator.company
ORCID: 0009-0006-0489-7881
DOI: 10.5281/zenodo.21995981
Discuss the paper with Gemini Notebook

18 August 2026

Abstract

This paper defines an Actor Graph (AG) as a named mathematical object with pluggable execution semantics. Its central contribution is **triple-identity accountable mediation**: a persistent structural relation carries a LinkID, each recognised occurrence carries a TransactionID, and the reusable mediator actor carries an ActorID. Link and transaction references preserve Local versus OpaqueExternal constructors. Identity-faithful representation is defined by recoverability of this sorted signature, and a non-collapse theorem formalises the independent discriminators required by the link/occurrence/mediator identity split.

Account mutation is transaction-sourced. Typed account state is a dependent sum; every account type has a deterministic effect action, each conserved type additionally has an abelian effect group and equivariant valuation, and a successful commit couples canonical record creation, typed history append, account postings, and causal events. This yields deterministic state, order independence for commuting effects over causal linear extensions, closed-boundary conservation, and an explicit open-boundary flow identity. Recursive structure is given by a finitary polynomial functor and its final coalgebra. Raw finite observations are adequate for identity-preserving bisimulation under C0–C7, while registry-backed back-reference compression is proved adequate under S1–S4.

At the resolved accountability layer, ActorID, LinkID, TransactionID, AccountCoordinate, and PostingOccurrence sorts form an ID-normal skeleton. Its instance category is a presheaf topos with pointwise limits and colimits, adhesive structural gluing, and a categorical obstruction to reconstruction after identity collapse. Identity-preserving adjacency stores (LinkID, EdgeActorID) pairs; mediator-only and numeric matrices are explicit lossy projections. Closure under graph-valued continuation is proved at the signature level and interpreted, without a geometric claim, as structural self-similarity. Programming-language macro-expressibility and relative maximality remain open programmes.

Keywords: Actor Graphs, actor-valued edges, triple-identity accountable mediation, typed link references, transaction-sourced accounts, coinductive disclosure, identity-preserving bisimulation, presheaf accountability skeleton, signature closure.

Contents

1 Introduction **5**
1.1 Novelty kernel: triple-identity accountable mediation . . . 5
2 Scope and terminology **6**
3 Canonical schemes and visual notation **8**
3.1 Triple-identity accountable mediation . . . 8
3.2 Canonical and parallel actor-valued mediation . . . 8
3.3 Roles of one universal type . . . 9
4 Structural definition **10**
4.1 Definition of an Actor Graph . . . 10
4.2 Vertex and edge roles . . . 11
4.3 Ordinary graph projection . . . 11
4.4 Faithful typed incidence encoding . . . 11
5 The enriched Actor Graph **12**
5.1 The actor contract . . . 12
5.2 Transaction occurrences and mandatory link binding . . . 13
5.3 Triple-identity separation . . . 14
5.4 Typed accounts and effect algebra . . . 15
5.5 Transaction commit . . . 16
5.6 Balanced transactions and accounting boundaries . . . 16
5.7 Account-coherent disclosure . . . 17
5.8 Process forms . . . 18
5.9 Potential and realised topology . . . 18
5.10 Enriched form . . . 19
5.11 Internal and external actors . . . 20
6 Coinductive recursive structure **20**
6.1 Graph-valued actor processes . . . 20
6.2 Recursive-reference relation and cyclicity . . . 21
6.3 The Actor Graph signature functor . . . 21
6.4 Final coalgebra and well-formed Actor Graphs . . . 22
6.5 From identity-referenced specifications to coalgebras . . . 23
6.6 Finite-depth observations . . . 24
6.7 Identity-preserving bisimulation . . . 25
6.8 Functoriality of finite disclosure . . . 26
6.9 Delegation cycles . . . 27
6.10 Depth of disclosure . . . 28
7 Categorical identity-incidence-accountability skeleton **28**
7.1 Resolved disclosure views and ID-normal form . . . 28
7.2 The Actor Graph accountability schema . . . 29
7.3 Categorical non-reconstructability . . . 31
8 Structural fractality **32**
8.1 Lambda-calculus reading of Actor Graph self-application . . . 32
8.2 Edge fractality . . . 33
8.3 Disclosure notation and instance-level witnesses . . . 33
8.4 Structural, not geometric, self-similarity . . . 34
9 Layers and graph types **34**
10 Matrix representations **35**
10.1 Vector-valued adjacency matrix . . . 36
10.2 Semiring path interpretation . . . 37
10.3 Binary adjacency projection . . . 37
10.4 Multiplicity matrix . . . 37
10.5 Source and target incidence matrices . . . 37
10.6 Relationship between adjacency and incidence . . . 38
10.7 Loss of identity under numeric projection . . . 38
10.8 Degree matrices . . . 38

10.9 Link-adjacency matrix . . . 39
10.10 Transaction-adjacency matrix . . . 39
10.11 Transaction incidence matrices . . . 39
10.12 Account-posting matrices . . . 40
10.13 Paths, reachability, and distance . . . 41
10.14 Laplacian projections . . . 41
10.15 Weighted matrices . . . 41
10.16 Current-state adjacency . . . 41
10.17 Potential adjacency . . . 41
10.18 Block matrices for layers . . . 42
10.19 Recursive-reference matrix . . . 42
10.20 Configuration-transition matrix . . . 42
10.21 Matrix hierarchy . . . 42
11 Worked examples 43
11.1 Transaction, accounts, and reversal . . . 43
11.2 Minimal cyclic Actor Graph . . . 44
12 Execution semantics 44
13 Active Transaction Graphs as a strict specialisation 45
14 Relation to adjacent formalisms 46
14.1 Property graphs and RDF 1.2 . . . 46
14.2 Object-Role Modeling . . . 47
14.3 UML association classes and SysML association blocks . . . 47
14.4 TypeDB and nested relations . . . 47
14.5 Reo and behavioural connectors . . . 47
14.6 Metagraphs, ubergraphs, and recursive hypergraphs . . . 48
14.7 Higraphs and hierarchical graph transformation . . . 48
14.8 Attributed graph transformation . . . 48
14.9 Semiring matrices and weighted automata . . . 48
14.10 The REA accounting ontology and OeBTO . . . 48
14.11 DEMO and Enterprise Ontology . . . 49
14.12 Algebra of double-entry bookkeeping . . . 49
14.13 Classical graph theory . . . 49
14.14 The Actor Model . . . 49
14.15 Petri nets . . . 50
14.16 Bigraphs and mobile systems . . . 50
14.17 Statecharts, DEVS, and Ptolemy II . . . 50
14.18 Process mining . . . 50
14.19 Presheaf and adhesive graph categories . . . 50
14.20 Comparative boundary . . . 51
15 Actor Graphs and programming-language expressiveness 53
15.1 Computability is not the axis . . . 53
15.2 Lambda calculus as the local computational substrate . . . 53
15.3 A Felleisen-style criterion for Actor Graph embeddings . . . 54
15.4 Formal anchor and the remaining separation problem . . . 54
15.5 Limit expressiveness and representational absorption . . . 55
15.6 Where code-primary systems reconstruct the missing invariants . . . 56
15.7 Model-primary construction . . . 56
16 Contributions and claimed scope 57
17 Limitations and open problems 58
17.1 Execution and commit semantics . . . 58
17.2 Spatial disclosure versus temporal execution . . . 58
17.3 Behavioural quotients . . . 58
17.4 Process-aware path algebra . . . 59
17.5 Infinite histories and finite analysis . . . 59
17.6 Identity governance . . . 59
17.7 Reuse of edge actors . . . 59

17.8 Fractal invariants . . . 59
17.9 Scale and storage . . . 59
17.10 Valuation, conversion, and non-quantitative accounts . . . 59
17.11 Programming-language separation . . . 59
17.12 Model-primary construction . . . 60
17.13 Relative maximality and software absorption . . . 60
17.14 Full enriched composition and categorical completion . . . 60
17.15 Empirical validation . . . 61
18 Conclusion **61**
Data and source availability **62**
Disclosure of interest **62**
Selected related publications by the author **62**
References **63**

1 Introduction

Graphs conventionally separate a system into vertices and edges. Vertices carry identity or state; edges express relations. This distinction is mathematically productive, but it becomes a modelling limit when the relation is itself an operating entity. A payment network, contract, approval route, logistics channel, access-control mediator, API gateway, or institutional procedure may have its own identity, state, transaction history, rules, risks, and accounting consequences.

One can reify such a relation by inserting another vertex. Reification is useful, and this paper does not claim that Actor Graphs cannot be encoded by ordinary labelled graphs. The stronger question is what the **native signature** treats as primitive. If the concrete structural relation, each occurrence through it, and the reusable mediator performing it have separate identities; if the mediator has the same accountable and process-bearing contract as its endpoints; and if it can disclose into another graph while retaining identity, then a simple mediator vertex captures the object only after additional node types, incidence roles, identity constraints, transaction identities, account attribution, and recursive-disclosure rules are supplied externally.

This paper defines that native object under the name **Actor Graph** (AG):

> A graph node is an actor; every realised edge role is performed by an actor of the same universal type while the structural link retains a distinct identity; every actor has accounts, transactions, events, and process; and the process of any actor may refer to another Actor Graph of the same class.

The architectural basis was disclosed in the *Actor Graph Engine* patent application. It replaces ordinary graph nodes with actors and graph edges with edge actors, distinguishes source actor, target actor, `Link ID`, and `Edge Actor`, gives actors account trees with plan/fact and debit/credit coordinates, shows transaction events and double-entry changes produced by actor interactions, and permits an actor function to be a process, another actor, or another Actor Graph [1]. It also distinguishes virtual null edges, realised null edges, and functional edge actors and presents graph layers, graph types, and recursion without a prescribed upper bound on levels [1].

1.1 Novelty kernel: triple-identity accountable mediation

The structural commitments developed below instantiate one compositional claim. For a realised link λ and a transaction occurrence τ executed through it, Actor Graphs preserve three independent identities:

$$\ell(\lambda) \in \mathsf{LinkID}, \qquad \theta(\tau) \in \mathsf{TransactionID}, \qquad \iota(\varepsilon(\lambda)) \in \mathsf{ActorID}.$$

These identities denote, respectively:

1. the persistent **structural relation**;
2. one recognised **transaction occurrence** through that relation;
3. the reusable **mediator** performing the relation.

The identity spaces are pairwise disjoint. A link may carry many transactions; one mediator may perform many links; one ordered actor pair may carry several realised links with distinct or reused mediators; and a transaction is bound to exactly one realised or explicitly opaque external link. The mediator carries the uniform contract

$$\langle \text{identity}, \text{accounts}, \text{transactions}, \text{events}, \text{process} \rangle$$

and may disclose into another Actor Graph while preserving its ActorID and the attribution of account state and transaction history. This conjunction is called **triple-identity accountable mediation** in this paper. It is the shortest statement of the novelty claim.

The claim is deliberately narrow. The paper does not claim priority for reification, recursive graphs, active components, accounting ontologies, double-entry algebra, event histories, actor

systems, hypergraphs, hierarchical simulation, or semiring-valued matrices considered separately. It claims a native composition in which relation, occurrence, and performing party remain distinct; the performing relation is a full accountable actor; and recursive disclosure preserves identity and accountable continuity.

Two further commitments complete the object. First, account state is transaction-sourced: no committed account mutation exists without an identified transaction occurrence, while events preserve the causal evidence of the commit. Second, recursive graph references may be cyclic. Actor Graph specifications store stable identity references, finite-depth observations disclose any selected amount of structure, and equality at the recursive boundary is expressed by identity-preserving bisimulation.

The paper now also isolates the resolved categorical layer of this object. A resolved disclosure view is normalised to five sorts—actors, links, transactions, account coordinates, and posting occurrences—connected by source, mediator, target, transaction-binding, account-ownership, and posting-attribution maps. These structures form a many-sorted functor category. The resulting presheaf characterisation is not a claim that every policy, history, registry, or recursive process has become elementary topos data. It is a precise theorem about the **identity-incidence-accountability skeleton**, together with an explicit boundary around the richer enriched and coalgebraic layers.

The paper proceeds from scope, notation, and canonical schemes to the structural object, typed reification, the enriched actor contract, effect and balance algebra, the explicit signature functor, coinductive recursion, finite observations, the categorical identity-incidence-accountability skeleton, structural fractality, matrix families, execution semantics, transactional specialisation, and related-work boundaries. Active Transaction Graphs (ATGs) are positioned as a stricter transactional closure of the general Actor Graph rather than as the first place where transactions appear.

A final section places the same object on the programming-language expressiveness axis. The claim is not that Actor Graphs compute functions unavailable to conventional Turing-complete languages. Following Felleisen, the relevant question is whether the Actor Graph invariants can be eliminated by a local, syntax-directed translation or whether faithful implementation requires a transformation of the surrounding program and execution substrate [33]. The section also makes explicit the relation to lambda calculus: lambda terms and fixed-point recursion remain available inside `LocalProcess`, while the Actor Graph adds persistent identities, accountable state, recognised transactions, causal evidence, and recursive disclosure around that local computational core. In this calibrated sense, Actor Graph is proposed as a candidate limit of native expressiveness for operational-system modelling: classical programming is not abolished but represented and absorbed as an internal layer of the executable model.

2 Scope and terminology

The phrase *actor graph* has appeared in earlier literature in several different senses. Actor garbage-collection research uses an actor graph or actor-reference graph to represent actors connected by references for liveness analysis [23]. Executable conceptual-graph research uses *actor graph* for procedural constructs built from actor and concept types [24]. Other fields use the phrase descriptively for graphs whose nodes represent human or software actors. These are legitimate prior uses of the words, but they do not define the object introduced here: their edges are references, channels, arcs, conceptual relations, or data-set relations rather than actors of the same universal type as their endpoints, and they do not impose the triple identity or account/transaction contract used here.

Here, **Actor Graph** refers only to the object defined in this manuscript. The novelty claim is not ownership of the two-word phrase. It concerns the explicit native conjunction of:

- actor-valued edges;
- distinct link, transaction, and mediator-actor identities;
- one account-, transaction-, event-, and process-bearing actor contract across roles;

- coinductive graph-valued processes with identity-preserving observation;
- structural and account-coherent disclosure;
- native topology, occurrence, and posting representations.

The following terms are distinguished:

- **Actor Graph (AG):** the general graph model defined here.
- **Structural Actor Graph:** the identity and topology tuple, independent of a chosen execution discipline.
- **Enriched Actor Graph:** a structural AG together with actor accounts, transaction histories and records, events, processes, effect algebras, balance policies, and potential topology.
- **Actor Graph specification:** a finite or countable identity-referenced description that may contain cyclic graph references.
- **Actor Graph observation:** a finite-depth disclosure of such a specification.
- **Account-coherent disclosure:** a declared aggregation relation under which an outer actor account observation agrees with a selected disclosure of its internal Actor Graph.
- **Actor Graph Engine:** a mechanism for generating, executing, changing, and simulating Actor Graphs; the architecture is disclosed in US20240378338A1 [1].
- **Active Transaction Graph (ATG):** the strict transactional subclass of enriched AGs in which every formally recognised interaction is transaction-identified and execution additionally satisfies transaction primacy, replay, compensation, idempotency, balance requirements for declared conserved types, and `(result, trace, ledger)` invariants [2].
- **Actor Graph implementation:** a software environment that instantiates some or all of the model. Corezoid and Simulator.Company are implementations developed by the author's company; they are not part of the mathematical definition.

Two comparison relations used later are also reserved here. The notation

$$L_{\mathrm{AG}} \preceq_{\mathrm{macro}} L$$

means that the declared Actor Graph extension is macro-expressible in host language L under Definition 24. The notation

$$\mathsf{ClassicalProgramming} \preceq_{\mathrm{model}} \mathsf{ActorGraphModeling}$$

is an interpretive model-inclusion claim: conventional programs may inhabit the `LocalProcess` position of an Actor Graph model. It is not a computability ordering and is not included among the proved results.

The word **actor** is used more broadly than a runtime-resident actor object. An actor is a stably identifiable and operationally significant entity. It may be software, a human role, an organisation, a device, a document, a contract, a transaction actor, or an institutional mediator. The definition does not assert that every physical object must be represented as an actor. Actor status is a modelling decision determined by causal and operational significance at the selected disclosure scale.

A **link** and a **transaction** are distinct. A link is a realised structural relation that may persist across many occurrences. A transaction is one identified occurrence executed or recognised through such a relation. Hence the identifier spaces satisfy

$$\mathsf{LinkID} \cap \mathsf{TransactionID} = \mathsf{TransactionID} \cap \mathsf{ActorID} = \mathsf{ActorID} \cap \mathsf{LinkID} = \emptyset.$$

The core model is a mathematical object with **pluggable execution semantics**. It fixes actor identity, roles, topology, account and transaction carriers, effect and balance invariants, recursive references, and representational constraints. It does not force one mailbox, scheduling, concurrency, consensus, stochastic, or institutional execution discipline.

3 Canonical schemes and visual notation

The Actor Graph is not introduced by matrices. Its primary intuition is topological, transactional, and multi-scale: an actor may occupy a vertex role, an edge role, or a graph-actor role while preserving one actor identity; a persistent link and each occurrence through it retain separate identities; and the same Actor Graph constructor may reappear inside any actor. The patent discloses this structure through actor anatomy, account trees, transaction events, edge actors, connection rules, edge types, graph structures, layers, and recursion [1].

3.1 Triple-identity accountable mediation

Figure 1 isolates the novelty kernel. A structural link, a transaction occurrence, and the actor performing the relation are distinct objects. Only the mediator is an actor, but it is a full actor: it owns accounts and transaction history, emits events, executes a process, and may recursively disclose into another Actor Graph.

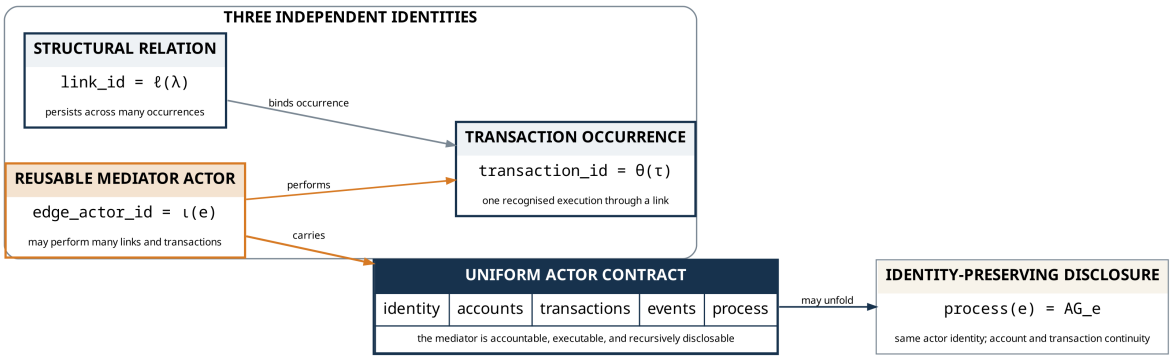


Figure 1: Triple-identity accountable mediation. `link_id` identifies the structural relation, `transaction_id` identifies one occurrence, and `edge_actor_id` identifies the reusable accountable mediator.

3.2 Canonical and parallel actor-valued mediation

Figure 2(a) shows the canonical actor-valued link. A realised link has its own `link_id`, while the relation is performed by an actor with its own `actor_id`. Source, mediator, and target actors belong to one actor universe.

One ordered pair of vertex-role actors may be connected by several realised links simultaneously, and the multiplicity has two structurally different forms. First, each link may be performed by its own edge actor: the split-tender example in Figure 2(b) carries a monetary leg through payment actor E_1 and a loyalty-points leg through loyalty-programme actor E_2, with distinct links and value types in the sense of Definition 5. Second, one reusable edge actor may perform several persistent links: the standard and express channels in Figure 2(c) are distinct links because their enduring fees, latency, and limits differ, even though both are performed by the same payment actor E. This is not decorative generality. The reused-mediator case is precisely the witness used by Theorem 1, item 1: link identity cannot be recovered from mediator identity. Definition 22 therefore stores the ordered vector of all edge-actor identities in one adjacency cell, including repeated ActorID values when one mediator performs several links. Each individual link may in turn host many transaction occurrences, so mediator, link, and transaction multiplicities are pairwise independent. In the reused-mediator case, the shared mediator retains one ActorID and one actor-level account and transaction carrier across both links; an execution specialisation may partition these internally. Potential topology has the same shape: Definition 10 and the potential-adjacency construction may admit several alternative mediators for one ordered actor pair.

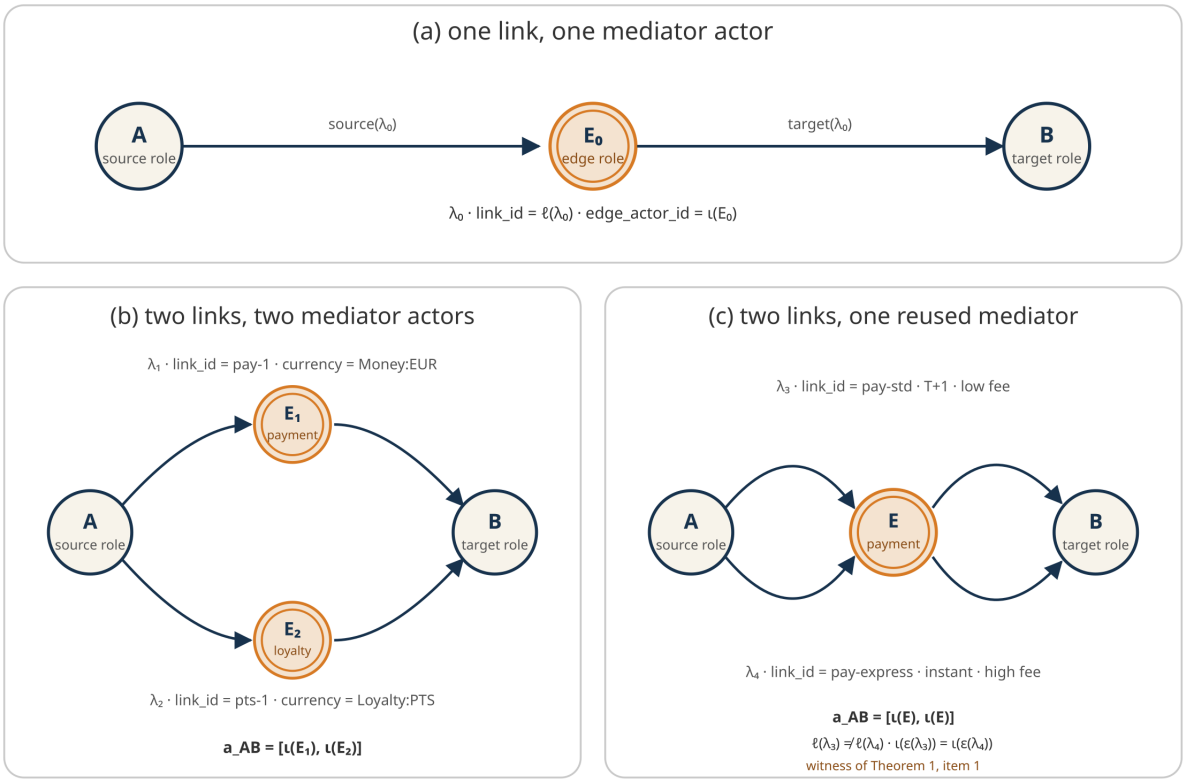


Figure 2: Canonical and parallel actor-valued mediation. (a) One realised link with distinct `link_id` and `edge_actor_id`. (b) Two parallel links use distinct mediators and value types, giving $\mathbf{a}_{AB} = [\iota(E_1), \iota(E_2)]$. (c) Two distinct links reuse one mediator, so $\ell(\lambda_3) \neq \ell(\lambda_4)$ while $\iota(\varepsilon(\lambda_3)) = \iota(\varepsilon(\lambda_4))$, giving the repeated-identity cell $\mathbf{a}_{AB} = [\iota(E), \iota(E)]$. Each link may host many transaction occurrences.

3.3 Roles of one universal type

Figure 3 shows that vertex actor, edge actor, and graph actor are not disjoint ontological classes. They are roles that the same universal actor type may occupy in different graph contexts or at different scales.

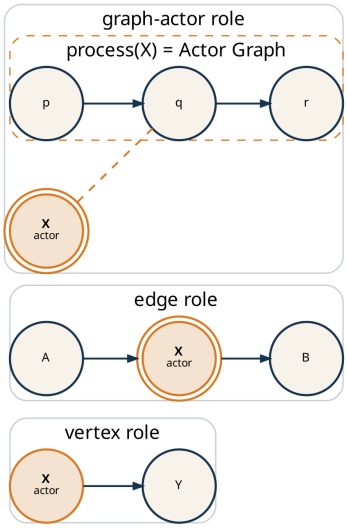


Figure 3: One universal actor type occupying vertex, edge, and graph-actor roles.

These schemes are explanatory projections, not extra entities added to the formal object. Definition 1 gives their common structure.

4 Structural definition

Let $\mathbb{A}$ be a universe of actor records. Let ActorID, LinkID, and TransactionID be pairwise disjoint sets of actor, link, and transaction identifiers. Assume canonical strict total orders

$$\prec_A, \qquad \prec_L, \qquad \prec_T$$

on the three identifier sets. Let

$$\iota : \mathbb{A} \longrightarrow \mathsf{ActorID}$$

be an injective actor-identity map. A registry

$$\mathcal{R} : \mathsf{ActorID} \rightharpoonup \mathbb{A}$$

resolves stable identifiers to actor records and satisfies $\mathcal{R}(\iota(a)) = a$ wherever the actor is locally resolved. Actor Graph specifications store stable references to ActorID; they do not require literal recursive copying of actor records.

4.1 Definition of an Actor Graph

Definition 1 (structural Actor Graph). A structural Actor Graph is a tuple

$$G = (A, \Lambda, s, t, \varepsilon, \ell),$$

where:

- $A \subseteq \mathbb{A}$ is a set of actors;
- Λ is a set of realised directed links;
- $s : \Lambda \to A$ maps each link to its source actor;
- $t : \Lambda \to A$ maps each link to its target actor;
- $\varepsilon : \Lambda \to A$ maps each link to the actor that performs the edge role;
- $\ell : \Lambda \longrightarrow \mathsf{LinkID}$ is an injective link-identity map.

Each link $\lambda \in \Lambda$ is written

$$\lambda = \langle \ell(\lambda), s(\lambda), \varepsilon(\lambda), t(\lambda) \rangle,$$

and graphically

$$s(\lambda) \xrightarrow{\varepsilon(\lambda)} t(\lambda).$$

The defining condition is

$$s(\lambda), t(\lambda), \varepsilon(\lambda) \in A.$$

Thus source, mediator, and target actors belong to the same actor universe.

The distinction

$$\ell(\lambda) \neq \iota(\varepsilon(\lambda))$$

is one of type and meaning, not necessarily a comparison of literal encodings. The first value identifies a concrete realised connection. The second identifies the actor performing the relation. One edge actor may be reused in several links, while every link remains independently identifiable.

4.2 Vertex and edge roles

Let

$$A_{\text{iso}} = A \setminus (\text{im}(s) \cup \text{im}(t) \cup \text{im}(\varepsilon))$$

be the actors occupying no source, target, or edge role in a realised link at the selected disclosure level.

Definition 2 (role sets). The vertex-role and edge-role sets are

$$V(G) = \text{im}(s) \cup \text{im}(t) \cup A_{\text{iso}},$$
$$E_A(G) = \text{im}(\varepsilon).$$

In general,

$$V(G) \cap E_A(G) \neq \emptyset.$$

An actor may occupy a vertex role in one relation and an edge role in another. Parallel links and self-links are permitted.

4.3 Ordinary graph projection

Forgetting the actor assigned to each edge gives the directed multigraph

$$\pi_V(G) = (V(G), \Lambda, s, t).$$

Remark 1 (forgetful projection). $\pi_V(G)$ preserves source-target connectivity but forgets ε. Actors that occur only in $E_A(G) \setminus V(G)$ disappear entirely from the vertex carrier. Their accounts, transaction histories, events, process, and reuse across links are not reconstructible from $\pi_V(G)$ alone.

4.4 Faithful typed incidence encoding

Actor Graphs can be encoded as ordinary labelled graphs, but a faithful encoding must preserve the distinction among actor nodes, link occurrences, and incidence roles.

Definition 3 (typed incidence encoding). For a structural Actor Graph G, define $L(G)$ with node set

$$N_L = A \sqcup \Lambda.$$

Actor nodes are labelled by type `Actor` and by $\iota(a)$. Link nodes are labelled by type `Link` and by $\ell(\lambda)$. For every $\lambda \in \Lambda$, add three typed arcs:

$$s(\lambda) \xrightarrow{\text{source}} \lambda,$$
$$\lambda \xrightarrow{\text{mediator}} \varepsilon(\lambda),$$
$$\lambda \xrightarrow{\text{target}} t(\lambda).$$

Figure 4 places the native link beside its faithful typed incidence encoding.

Proposition 1 (faithfulness of typed incidence encoding). The construction L is injective up to type-, role-, and identity-preserving graph isomorphism.

Proof. Actor nodes and link nodes are recovered from their disjoint node types. Their identifiers recover ι and ℓ. For each link node, the unique arcs labelled `source`, `mediator`, and `target` recover $s(\lambda)$, $\varepsilon(\lambda)$, and $t(\lambda)$. Hence the full structural tuple is reconstructible. □

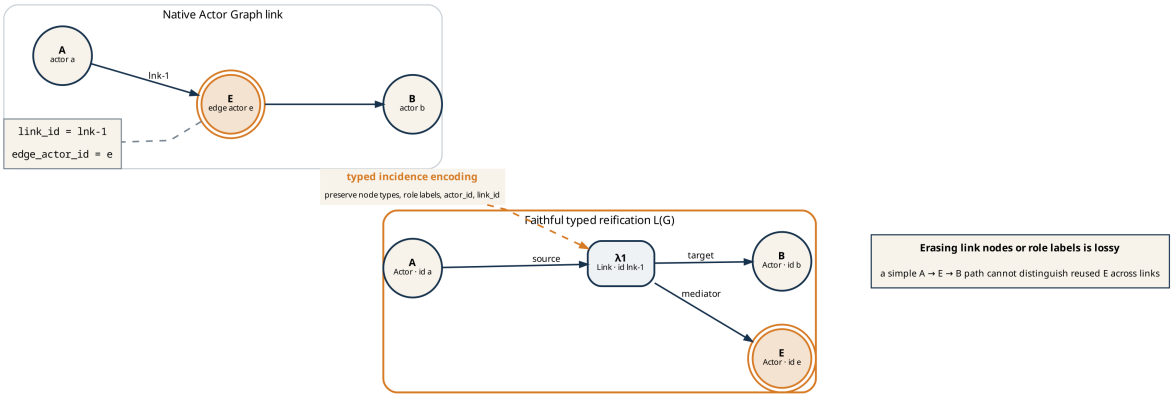


Figure 4: Native Actor Graph link and faithful typed incidence encoding. The concrete link identity is shown as `lnk-1`; the encoding is reversible only while actor/link node types, source/mediator/target role labels, `actor_id`, and `link_id` are preserved.

Corollary 1 (loss under untyped reification). The simpler path replacement

$$s(\lambda) \longrightarrow \varepsilon(\lambda) \longrightarrow t(\lambda)$$

is not generally injective. If the same edge actor is reused in two links, erasing link nodes or incidence-role labels can collapse distinct link identities into one mediator path. Actor Graph novelty therefore does not rest on non-encodability; it rests on treating this typed, identity-preserving signature as native rather than as an external convention over a poorer graph.

5 The enriched Actor Graph

The structural tuple identifies topology and roles. The enriched model adds the canonical actor contract, mandatory transaction-to-link binding, typed account effects, balance policies, account-coherent disclosure, and potential topology.

5.1 The actor contract

Figure 5 summarises the universal actor contract and the four coupled obligations of a successful commit.

Let Accounts, Events, and LocalProcess denote suitable domains of attributable account state, event histories, and local processes. Let $\mathsf{TxHistory} = \mathsf{List}_f(\mathsf{TransactionID})$ be the set of finite transaction-identity histories at one finite configuration.

Definition 4 (actor contract). Every actor $a \in A$ has a contract

$$\boxed{\mathcal{C}(a) = \langle \iota(a), \mathrm{accounts}(a), \mathrm{transactions}(a), \mathrm{events}(a), \mathrm{process}(a) \rangle}$$

with the following roles:

- **actor identity** $\iota(a)$: continuity across graph contexts, runtime addresses, process implementations, and disclosure scales;
- **accounts:** typed state attributable to the actor, including balances, plan/fact values, debit/credit coordinates, limits, rights, obligations, probabilities, quotas, or other accountable variables;
- **transactions:** an ordered append-only history of `transaction_id` values relevant to the actor;
- **events:** causal and temporal evidence, including external, internal, and where required expected future events;

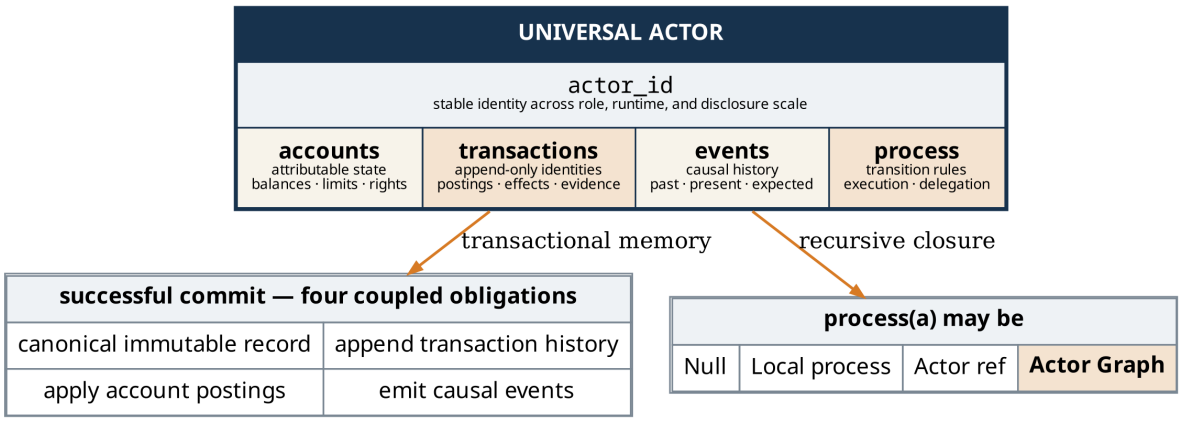


Figure 5: Canonical actor anatomy. Identity, accounts, transactions, events, and process form one operational actor. The commit strip shows all four coupled obligations: preserve a canonical immutable record, append transaction history, apply account postings, and emit causal events; the process may itself reference another Actor Graph.

- **process:** the local rule, delegation, or graph reference by which the actor recognises interactions, validates preconditions, commits transactions, updates accounts, emits events, or recognises externally governed transitions.

The patent architecture expressly includes actor account trees, plan/fact and debit/credit coordinates, transaction events created by actor interactions, and double-entry changes to the accounts of interacting and service actors [1]. The contract preserves those mechanisms as part of the general Actor Graph anatomy.

5.2 Transaction occurrences and mandatory link binding

A structural link can persist while many transaction occurrences pass through it.

Definition 5 (Actor Graph transaction occurrence). Let Θ be a set of transaction occurrences and

$$\theta : \Theta \longrightarrow \mathsf{TransactionID}$$

an injective transaction-identity map. Every recognised transaction $\tau \in \Theta$ is bound to exactly one realised link $\lambda \in \Lambda$, or to exactly one explicitly opaque external `link_id` when the relation belongs to another jurisdiction. Its canonical record is

$$\begin{aligned}\mathcal{T}(\tau) = \langle \theta(\tau), \mathrm{link_id}(\tau), \mathrm{source_actor_id}(\tau),\\ \mathrm{edge_actor_id}(\tau), \mathrm{target_actor_id}(\tau), \mathrm{value_type}(\tau),\\ \mathrm{value}(\tau), \mathrm{protocol}(\tau), \mathrm{preconditions}(\tau),\\ \mathrm{metadata}(\tau), \mathrm{effects}(\tau)\rangle.\end{aligned}$$

For a locally realised link λ, binding requires

$$\mathrm{link_id}(\tau) = \ell(\lambda),$$

$$\mathrm{source_actor_id}(\tau) = \iota(s(\lambda)), \qquad \mathrm{edge_actor_id}(\tau) = \iota(\varepsilon(\lambda)), \qquad \mathrm{target_actor_id}(\tau) = \iota(t(\lambda)).$$

Thus the `link_id` field identifies a reusable structural relation; `transaction_id` identifies one occurrence through that relation; and `edge_actor_id` identifies the reusable mediator performing it. An external occurrence may keep the same record shape while its link and actor identities remain opaque but stable.

The `value_type` field declares the type or denomination of the attributable or transferred value. Currency is one supported value type. Financial implementations may retain `currency` as

a compatibility alias. The `effects` field contains typed account postings and any other declared state effects.

5.3 Triple-identity separation

Define the mediator-identity map

$$\mu = \iota \circ \varepsilon : \Lambda \to \mathsf{ActorID}.$$

The maps

$$\ell : \Lambda \longrightarrow \mathsf{LinkID}, \qquad \theta : \Theta \longrightarrow \mathsf{TransactionID}, \qquad \mu : \Lambda \to \mathsf{ActorID}$$

identify structural relations, transaction occurrences, and relation-performing actors. The first two maps are injective; μ need not be, because one edge actor may perform several links.

For an enriched Actor Graph $\mathcal{G}$, let its **identity observation signature** be the sorted structure

$$\mathcal{O}_{\mathrm{id}}(\mathcal{G}) = (A, \Lambda, \Theta, \iota, \ell, \theta, \mu, \mathrm{bind}, s, t, \varepsilon),$$

where

$$\mathrm{bind} : \Theta \to \mathsf{LinkID}, \qquad \mathrm{bind}(\tau) = \mathrm{link_id}(\tau).$$

An isomorphism of identity observation signatures preserves the three identifier sorts, the source/mediator/target incidence maps, and transaction-to-link binding.

Definition 6 (identity-faithful representation). Let $\mathcal{C}_{AG}$ be a class of enriched Actor Graphs and let

$$E : \mathcal{C}_{AG} \to \mathcal{R}_E$$

be a representation into an arbitrary representation domain. The representation E is **identity-faithful** when there exists a recovery map on its image,

$$D_E : E(\mathcal{C}_{AG}) \longrightarrow \{\mathcal{O}_{\mathrm{id}}(\mathcal{G}) \mid \mathcal{G} \in \mathcal{C}_{AG}\},$$

such that for every $\mathcal{G} \in \mathcal{C}_{AG}$,

$$D_E(E(\mathcal{G})) \cong_{\mathrm{id}} \mathcal{O}_{\mathrm{id}}(\mathcal{G}).$$

Equivalently, whenever $E(\mathcal{G}) = E(\mathcal{H})$, the two identity observation signatures are identity-preserving isomorphic. The term *faithful* in Theorem 1 refers to this recoverability condition, not merely to the ability of a host formalism to store some encoding.

Theorem 1 (non-collapse of link, occurrence, and mediator identity). Consider the class of enriched Actor Graphs that permits parallel links, repeated transactions through one link, and reuse of one edge actor across links. No representation of the full class is identity-faithful if it identifies:

1. links solely by mediator ActorID;
2. transactions solely by their bound LinkID; or
3. transactions solely by mediator ActorID.

Equivalently, every identity-faithful representation of the full class must retain independent discriminators equivalent to link identity, transaction-occurrence identity, and mediator-actor identity, whether or not another formalism uses the names `link_id`, `transaction_id`, and `edge_actor_id`.

Proof. For item 1, choose distinct links $\lambda_1 \neq \lambda_2$ with $\mu(\lambda_1) = \mu(\lambda_2)$; a mediator-only link key

collapses them. For item 2, choose distinct occurrences $\tau_1 \neq \tau_2$ bound to the same link; a link-only transaction key collapses them. For item 3, choose two occurrences performed by the same reusable mediator, whether through one or several links; a mediator-only transaction key collapses them. □

Remark 2 (identity sorts under alternative encodings). The theorem does not claim that adjacent formalisms cannot encode these distinctions. It identifies three independent invariants of the Actor Graph signature. A faithful alternative encoding must therefore provide discriminators from which the corresponding link, occurrence, and mediator identity sorts are recoverable, regardless of the names used by the host formalism.

5.4 Typed accounts and effect algebra

Let AccountID be a set of account identifiers and Type a set of account-value types. For each $\kappa \in \mathsf{Type}$, declare:

- a value domain V_κ;
- an effect monoid $(\Delta_\kappa, \oplus_\kappa, 0_\kappa)$;
- a deterministic monoid action
$$\mathrm{apply}_\kappa : V_\kappa \times \Delta_\kappa \to V_\kappa;$$
- a flag $\mathrm{conserved}(\kappa) \in \{0, 1\}$.

The action satisfies

$$\mathrm{apply}_\kappa(v, 0_\kappa) = v,$$
$$\mathrm{apply}_\kappa(\mathrm{apply}_\kappa(v, d_1), d_2) = \mathrm{apply}_\kappa(v, d_1 \oplus_\kappa d_2).$$

The monoid need not be commutative for every non-quantitative domain; an execution specialisation must then preserve the declared commit order. If a type is marked conserved, Δ_κ is required to be an abelian group. For ordinary quantitative bookkeeping it may be obtained as the group of differences of an underlying commutative monoid, the algebraic construction used by Ellerman to formalise double-entry bookkeeping [30, 31]. This paper claims no novelty for that algebra; its contribution is where the algebra is placed in the graph.

Definition 7 (typed actor-account system). The account family of actor a is a finite partial map

$$\mathrm{accounts}(a) : \mathsf{AccountID} \rightharpoonup \coprod_{\kappa \in \mathsf{Type}} V_\kappa,$$

with typing map

$$\mathrm{type}_a : \mathrm{dom}(\mathrm{accounts}(a)) \to \mathsf{Type}.$$

The global key of an account is $(\iota(a), q)$. No account state exists without attribution to one stable ActorID; vertex actors, edge actors, graph actors, and external actors obey the same rule.

A posting is a typed record

$$p = \langle \mathrm{actor_id}(p), \mathrm{account_id}(p), \mathrm{type}(p), \delta(p) \rangle$$

with $\delta(p) \in \Delta_{\mathrm{type}(p)}$. Write $\mathrm{Post}(\tau)$ for the finite posting family of transaction τ, and $\mathrm{Post}_a(\tau)$ for the postings owned by actor a.

No off-ledger mutation is permitted: a committed account change must be the effect of a committed transaction occurrence. Events record the causal evidence of that commit; they are not substituted for transaction identity.

5.5 Transaction commit

A committed transaction τ transforms configuration c into c' by one coupled commit:

$$\text{transactions}_{c'}(a) = \text{transactions}_{c}(a) \cdot [\theta(\tau)]$$

for every participating or posting-owning actor a,

$$\text{accounts}_{c'}(a) = \text{apply}(\text{accounts}_{c}(a), \text{Post}_{a}(\tau)),$$

and

$$\text{events}_{c'}(a) = \text{events}_{c}(a) \cdot \text{Event}_{a}(\tau).$$

Actors unaffected by τ retain their prior components. Transaction identity, account effect, and event evidence therefore refer to the same occurrence. Reversal, cancellation, and compensation do not erase the original transaction. They are new transactions with their own identities, causal references, and effects.

The transactional memory of an actor is

$$\text{TransactionalMemory}(a) = \text{Accounts}(a) + \text{TransactionHistory}(a),$$

while events preserve the causal trace by which that memory was produced.

5.6 Balanced transactions and accounting boundaries

Let $B \subseteq \text{ActorID}$ be a declared accounting boundary. For conserved type κ, let $\partial_B^\kappa(\tau) \in \Delta_\kappa$ be an explicit boundary effect. It is neutral for a closed boundary and records value entering or leaving through an external, settlement, equity, or other declared boundary account when the view is open. Boundary effects are not out-of-band metadata: they are stored in $\text{effects}(\tau)$ as postings to declared boundary accounts. The term $\partial_B^\kappa(\tau)$ is the aggregate of those boundary-account postings and is separated from the internal sum below only to avoid double counting.

Definition 8 (balanced transaction). A transaction τ is balanced for conserved type κ relative to boundary B when

$$\bigoplus_{\substack{p \in \text{Post}(\tau) \\ \text{actor_id}(p) \in B \\ \text{type}(p) = \kappa}} \delta(p) \;\oplus_\kappa\; \partial_B^\kappa(\tau) = 0_\kappa$$

For ordinary signed numeric postings this reduces to a zero column sum. The source, mediator, and target actors are the canonical interaction triple, but the balance test includes **all** postings of the transaction, including fees, taxes, reserves, insurers, settlement actors, and explicit boundary accounts.

The general Actor Graph core permits both conserved and non-conserved account types. Balance is mandatory whenever a type is declared conserved and the selected boundary is declared closed. Active Transaction Graph specialisations require this discipline for every declared conserved transaction type.

Proposition 2 (deterministic account state). Given an initial account state and, for each account (or actor), a total order on the committed transactions affecting it as supplied by the chosen execution specialisation, the account state at every observation is uniquely determined. No global total order over all actors is required.

Proof. Each account update is a deterministic monoid action. Folding the locally ordered effects therefore yields one value. □

Proposition 3 (order independence of commuting commits). If the per-account effects of two causally independent committed transactions commute, exchanging their order does not change

the resulting account state. Hence any two linearisations of a finite partial order that differ only by exchanges of commuting effects yield the same state.

Proof. Associativity follows from the action law; pairwise commutativity permits adjacent independent effects to be exchanged. Any two such linearisations are connected by a finite sequence of adjacent exchanges. □

Proposition 4 (local conservation). Within a closed boundary, a sequence of transactions balanced for conserved type κ preserves the aggregate κ-value.

Proof. Each committed transaction contributes the neutral group element; a finite composition of neutral elements is neutral. □

Figure 6 shows how one recognised transaction binds topology, postings, histories, and causal evidence at commit.

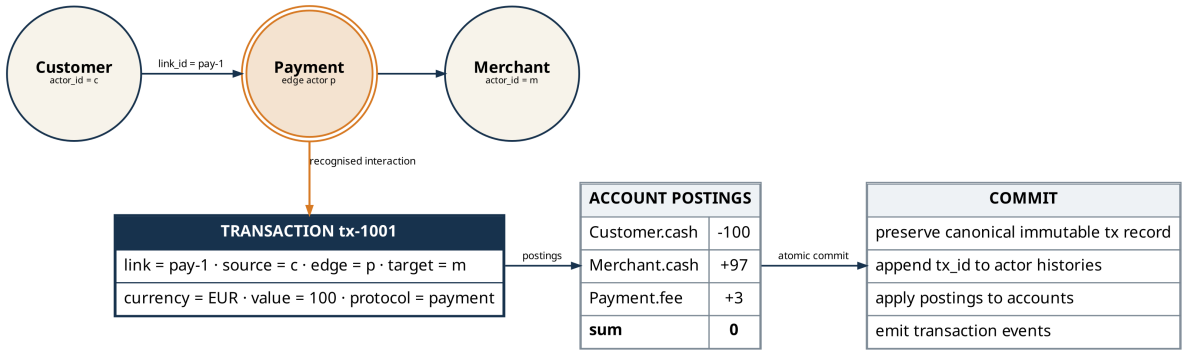


Figure 6: Accounts and transactions are coupled at commit: the transaction is bound to the link and edge actor, its canonical immutable record is preserved, histories append the transaction identity, postings update actor accounts, and transaction events preserve causal evidence.

5.7 Account-coherent disclosure

When an outer graph actor claims that its accounts summarise an internal graph, the relation must be explicit rather than assumed.

Let $A_D(G_a)$ be the actors visible in disclosure view D. The account family of that view is the ActorID-keyed disjoint union

$$\mathrm{Accounts}(G_a|_D) = \bigsqcup_{b \in A_D(G_a)} \{(\iota(b), q, \mathrm{accounts}(b)(q)) \mid q \in \mathrm{dom}(\mathrm{accounts}(b))\}.$$

Definition 9 (account-coherent disclosure). Let a be a graph actor with $\mathrm{process}(a) = G_a$, and let D be a declared finite disclosure view of G_a. An account-coherent disclosure consists of a declared partial aggregation map

$$\mathrm{agg}_{a,D} : \mathrm{Accounts}(G_a|_D) \rightharpoonup \mathrm{Accounts}(a)$$

such that, for every outer account coordinate claimed to summarise the view,

$$\mathrm{observe}_{\mathrm{outer}}(a) = \mathrm{agg}_{a,D}(\mathrm{observe}_{\mathrm{inner}}(G_a|_D)).$$

The map may be partial because private, external, or undisclosed accounts need not be consolidated. In a cyclic specification, coherence is stated for a finite disclosure view or for a domain-specific fixed-point aggregation, never by silently summing an infinite unfolding. Account-coherent disclosure preserves attribution while allowing one outer account to summarise many inner accounts.

5.8 Process forms

The process field is drawn from

$$\mathsf{Proc}(X) = \mathsf{Null} \sqcup \mathsf{LocalProcess} \sqcup \mathsf{ActorID} \sqcup X,$$

where X is a class of graph specifications. The four cases are:

- Null: no additional transformation;
- local process: direct execution or declarative transition rule;
- actor reference: delegation to another stable actor identity;
- graph reference: disclosure of another Actor Graph specification.

The notations

$$\mathrm{process}(a) = b$$

and

$$\mathrm{process}(a) = G_a$$

are shorthand for references to $\iota(b)$ and to a graph specification, not literal recursive record copying.

A connection without an additional function may be represented by a null actor. A functional connection is represented by an edge actor with a non-null process. Both remain actors and therefore retain accounts, transactions, events, and process. Figure 7 distinguishes the potential, realised-null, and functional cases.

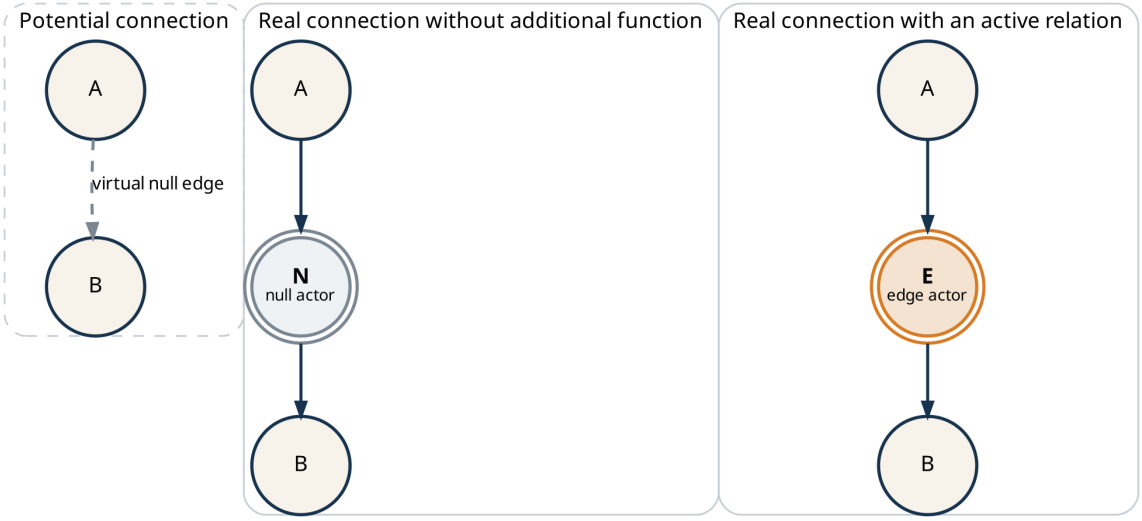


Figure 7: Potential, realised null, and realised functional actor edges.

5.9 Potential and realised topology

Definition 10 (potential-link relation). Let

$$\mathrm{mayLink} \subseteq A \times A \times A$$

be the set of admissible triples

$$(\mathrm{source}, \mathrm{mediator}, \mathrm{target}).$$

Every realised link must satisfy

$$(s(\lambda), \varepsilon(\lambda), t(\lambda)) \in \text{mayLink}\,.$$

If no restrictive policy is declared, the default is mayLink $= A^3$. An admissible triple with no member of Λ is potential topology. A realised link whose edge actor has null process is a realised null edge. A realised link whose edge actor has a non-null process is a realised functional edge.

This definition gives formal status to the patent distinction among virtual null edges, realised null edges, and realised actor edges [1], while also restricting which mediator may occupy the relation.

5.10 Enriched form

Definition 11 (enriched Actor Graph relative to a graph class X). An enriched Actor Graph is

$$\begin{aligned}\mathcal{G}_X = (&G, \mathcal{R}, \Theta, \mathcal{T},\\ &\text{accounts}, \text{transactions}, \text{events},\\ &\text{process} : A \to \mathsf{Proc}(X),\\ &\text{mayLink}, \text{balancePolicy}, \text{aggregationPolicy}).\end{aligned}$$

where G is a structural Actor Graph, Θ and $\mathcal{T}$ provide canonical transaction occurrences and records, and the remaining components provide actor resolution, account algebra, actor memory, causal evidence, graph-valued process references, potential topology, declared conservation boundaries, and optional account-coherent aggregation.

The well-formedness obligations deliberately separate two levels. The pairwise disjointness of the identifier universes is a signature-level axiom fixed globally:

$$\mathsf{ActorID} \cap \mathsf{LinkID} = \mathsf{LinkID} \cap \mathsf{TransactionID} = \mathsf{TransactionID} \cap \mathsf{ActorID} = \emptyset.$$

It is therefore not repeated as a state-local well-formedness condition.

Coalgebra-local conditions are predicates of one finite F_Σ-unfolding step:

- **C0. canonical carrier order:** the outer actor-record list is stored in ascending $\prec_A$-order; the realised-link list is stored in ascending $\prec_L$-order; the canonical-transaction-record list is stored in ascending $\prec_T$-order; and the potential-topology list is the duplicate-free lexicographically ordered enumeration induced by $\prec_A$ on source, mediator, and target ActorID values. Transaction-history lists inside actor records retain their declared commit order and are not sorted by identifier;
- **C1.** actor, link, and transaction identifiers are unique within their respective local lists;
- **C2.** every recognised transaction is bound to exactly one locally realised link or one explicitly typed opaque external link identity;
- **C3.** the `source_actor_id`, `edge_actor_id`, and `target_actor_id` fields in each locally realised transaction agree with its bound link;
- **C4.** every transaction identity in a local actor history resolves to the canonical transaction record in the same step or is explicitly marked opaque;
- **C5.** account effects name valid, correctly typed actor-account coordinates or declared boundary-account coordinates;
- **C6.** every transaction declared over a conserved type and closed boundary satisfies Definition 8;
- **C7.** every realised source/mediator/target triple belongs to mayLink.

Specification and execution side conditions range over more than one unfolding step or depend on external governance data:

- **S1.** the partial registry resolves every locally governed ActorID consistently across all reachable graph handles, and no stable identity is reassigned to a different actor;
- **S2.** configuration evolution is transaction-sourced: account mutation, canonical record creation, history append, and causal event emission obey the coupled commit rule;
- **S3.** every declared outer/inner account summary satisfies Definition 9 for its selected disclosure view and aggregation policy;
- **S4.** unresolved external ActorID, LinkID, and TransactionID values are governed by an explicit opaque-reference and jurisdiction policy.

An enriched specification is well formed when its one-step records satisfy C0-C7 and its registry, evolution, and disclosure policies satisfy S1-S4. Only C0-C7 participate in the invariant-subcoalgebra construction below; S1-S4 are admissibility conditions on identity-referenced specifications and their executions.

The coinductive graph class X is fixed in Definition 15.

5.11 Internal and external actors

An Actor Graph may include actors governed by different jurisdictions. A runtime actor may be executed locally. A human, bank, regulator, external service, or physical device may remain governed elsewhere. The external actor is outside the local execution jurisdiction but not outside the model.

Inclusion in the graph therefore does not imply local control. For an external actor, the process specifies the protocol by which the local graph recognises state claims, authority, evidence, and transactions produced by another jurisdiction. Unresolved external ActorID and LinkID values remain opaque under finite observations, are excluded from locally computed recursive-reference matrices unless explicitly resolved, and are not treated as malformed solely because the local registry is partial.

6 Coinductive recursive structure

6.1 Graph-valued actor processes

Definition 12 (graph actor). An actor a is a graph actor when its process is a reference to an Actor Graph G_a:

$$\text{process}(a) = G_a.$$

At the outer level, a remains one actor with identity $\iota(a)$, accounts, transaction history, events, and process. At the disclosed level, G_a describes the process of that same actor. An edge actor may also be a graph actor, so an outer relation

$$a_i \xrightarrow{e} a_j$$

may disclose

$$a_i \longrightarrow G_e \longrightarrow a_j.$$

The edge actor is not replaced by an unrelated graph. Its stable identity remains the outer handle for the disclosed process and its transaction and account history remains attributable to that identity. Figure 8 shows this disclosure using the canonical payment-request direction: gateway, acquirer, network, issuer.

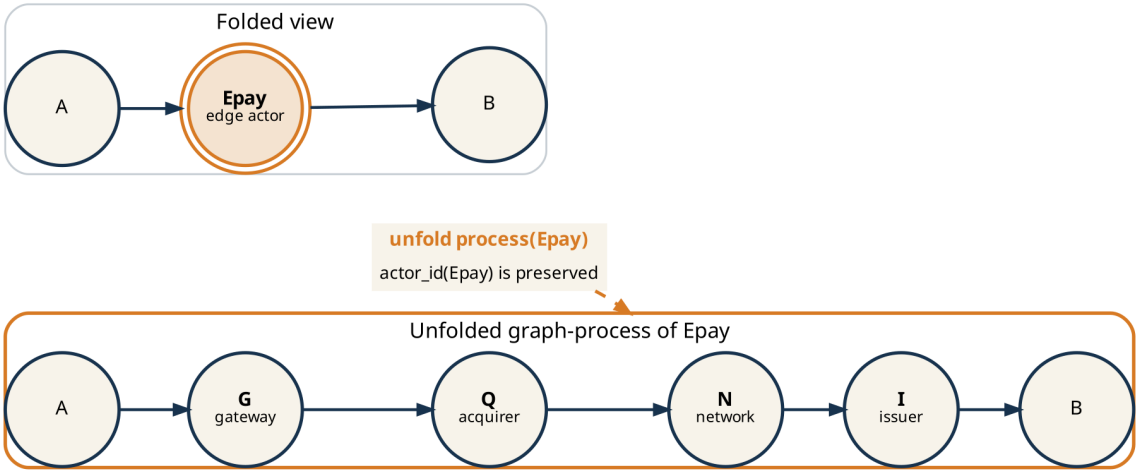


Figure 8: Identity-preserving disclosure of an edge actor. The payment-request path is gateway → acquirer → network → issuer, while `actor_id(Epay)` remains the outer identity.

6.2 Recursive-reference relation and cyclicity

For a graph H, define its actor-identity carrier

$$\mathrm{CarrierID}(H) = \{\iota(x) \mid x \in A(H)\}.$$

Definition 13 (recursive-reference relation). For actor identities $\alpha, \beta \in \mathsf{ActorID}$, write

$$\beta \prec_R \alpha$$

when $\mathcal{R}(\alpha)$ resolves to actor a, $\mathrm{process}(a) = G_a$, and

$$\beta \in \mathrm{CarrierID}(G_a).$$

No acyclicity or well-foundedness axiom is imposed. Both

$$\alpha \prec_R \alpha$$

and

$$\alpha \prec_R^+ \alpha$$

are admissible. The first is direct self-reference; the second is indirect cyclic self-reference. Because recursion is expressed through stable identity references, neither case requires a set to contain itself literally. Figure 9 shows direct and indirect cyclic references together with finite-depth observations.

6.3 The Actor Graph signature functor

Let

$$I = \mathsf{ActorID}, \qquad L = \mathsf{LinkID}, \qquad T = \mathsf{TransactionID}$$

and let Q, E, P, D be the sets of well-formed typed account states, event observations, local processes, and non-identity transaction data. The domain Q is restricted by Definitions 7 and 9 and the balance and aggregation policies of Definition 11. Define

$$\mathsf{Link}_\Sigma = L \times I \times I \times I,$$

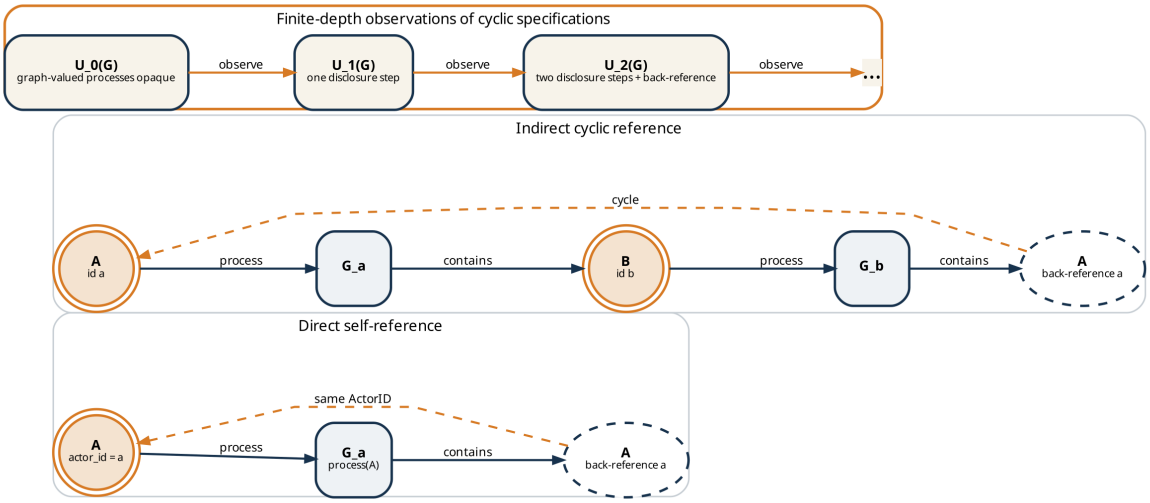


Figure 9: Coinductive recursion. Direct and indirect cyclic actor references are admissible; finite-depth observations remain finite for locally finite specifications and preserve back-references to stable ActorID values.

where the three ActorID coordinates are source, mediator, and target, and

$$\mathsf{Tx}_\Sigma = T \times L \times I \times I \times I \times D.$$

For a set X of graph-valued continuations, define

$$\mathsf{Proc}_\Sigma(X) = 1 + P + I + X$$

and

$$\mathsf{Actor}_\Sigma(X) = I \times Q \times \mathsf{List}_f(T) \times E \times \mathsf{Proc}_\Sigma(X).$$

The five actor coordinates are identity, accounts, transaction history, events, and process. The canonical transaction records are stored separately in Tx_Σ, so actor histories contain stable transaction references rather than duplicated records.

Definition 14 (Actor Graph signature functor). Define the endofunctor $F_\Sigma : \mathbf{Set} \to \mathbf{Set}$ by

$$\boxed{F_\Sigma(X) = \mathsf{List}_f(\mathsf{Actor}_\Sigma(X)) \times \mathsf{List}_f(\mathsf{Link}_\Sigma) \times \mathsf{List}_f(\mathsf{Tx}_\Sigma) \times \mathsf{List}_f(I \times I \times I)}$$

The four factors represent actor records, realised links, canonical transaction records, and potential source/mediator/target triples. On a map $f : X \to Y$, $F_\Sigma(f)$ acts as the identity on all constant coordinates and applies f only to graph-valued process positions.

Because

$$\mathsf{List}_f(X) \cong \coprod_{n \in \mathbb{N}} X^n,$$

F_Σ is a finitary polynomial endofunctor on Set.

6.4 Final coalgebra and well-formed Actor Graphs

Theorem 2 (existence of the coinductive carrier). The functor F_Σ admits a final coalgebra

$$\zeta : \nu F_\Sigma \longrightarrow F_\Sigma(\nu F_\Sigma).$$

Proof. The functor is built from constants, finite products, finite coproducts, and the finite-list functor. It is therefore a finitary polynomial endofunctor on Set. Standard final-coalgebra results for polynomial and finitary set functors apply [22, 26]; Worrell's general analysis places convergence of an arbitrary finitary set functor by stage $\omega + \omega$, which is sufficient for existence. □

The raw final coalgebra contains all records generated by the signature, including malformed ones. Let $\mathsf{WF}_{\mathsf{step}}$ denote the conjunction C0-C7 from Definition 11. It depends only on one F_Σ-unfolding step and is invariant under reindexing of recursive continuation positions. Hence the union of all invariant subcoalgebras whose states satisfy $\mathsf{WF}_{\mathsf{step}}$ is again an invariant subcoalgebra satisfying the same predicate. Define

$$\mathfrak{AG}_\Sigma \subseteq \nu F_\Sigma$$

as this largest step-well-formed invariant subcoalgebra. The side conditions S1-S4 are not silently folded into that construction: a rooted identity-referenced specification is **admissible** when its specification coalgebra lands in $\mathfrak{AG}_\Sigma$ and S1-S4 hold for its registry, execution history, opaque references, and declared aggregation maps.

Definition 15 (coinductive Actor Graph). A coinductive Actor Graph is a state in $\mathfrak{AG}_\Sigma$. This class includes finite acyclic specifications, finite cyclic specifications, and potentially infinite disclosure structures generated by recursive reference. It does not require a globally terminating full expansion.

6.5 From identity-referenced specifications to coalgebras

Let $\mathcal{S}$ be a rooted, locally finite Actor Graph specification satisfying S1-S4. A **graph handle** is either the distinguished root handle $r_{\mathcal{S}}$ or the stable ActorID of an actor whose process references another Actor Graph. Let $X_{\mathcal{S}}$ be the least set containing $r_{\mathcal{S}}$ and closed under all graph-valued process references reachable from the root. Local finiteness means that every resolved handle contains finite actor, link, transaction, and potential-topology lists; the carrier $X_{\mathcal{S}}$ itself may be finite or countably infinite.

For every handle $x \in X_{\mathcal{S}}$, resolve the corresponding enriched graph record, canonically order the outer actor, realised-link, canonical-transaction-record, and potential-topology carriers as required by C0, and encode the resulting one-step data in the four factors of F_Σ. Actor transaction histories retain their declared commit order. Every graph-valued process is replaced by the referenced handle in $X_{\mathcal{S}}$. This defines the **specification coalgebra**

$$\gamma_{\mathcal{S}} : X_{\mathcal{S}} \longrightarrow F_\Sigma(X_{\mathcal{S}}).$$

If graph-valued processes are indexed by the ActorID of their owning graph actor, then $X_{\mathcal{S}} \setminus \{r_{\mathcal{S}}\} \subseteq$ ActorID. Cyclic specifications simply produce cycles in $\gamma_{\mathcal{S}}$; they do not require recursive copying of records.

By finality there is a unique coalgebra morphism

$$[\![-]\!]_{\mathcal{S}} : (X_{\mathcal{S}}, \gamma_{\mathcal{S}}) \longrightarrow (\nu F_\Sigma, \zeta)$$

such that

$$\zeta \circ [\![-]\!]_{\mathcal{S}} = F_\Sigma([\![-]\!]_{\mathcal{S}}) \circ \gamma_{\mathcal{S}}.$$

The denotation of the rooted specification is

$$[\![\mathcal{S}]\!] = [\![r_{\mathcal{S}}]\!]_{\mathcal{S}}.$$

Naturality of the final-sequence maps gives

$$U_k(\mathcal{S}) = U_k^{X_{\mathcal{S}}}(r_{\mathcal{S}}) = U_k^{\nu F_\Sigma}([\![\mathcal{S}]\!]),$$

so the finite observations and identity-preserving bisimulation defined below apply continuously to the original identity-referenced specification. This construction closes the formal bridge from registry-backed specifications to their coinductive semantics.

6.6 Finite-depth observations

Consider an F_Σ-coalgebra

$$\gamma : X \to F_\Sigma(X).$$

Its final sequence begins

$$1 \longleftarrow F_\Sigma 1 \longleftarrow F_\Sigma^2 1 \longleftarrow \cdots.$$

Define the raw final-sequence maps

$$O_0^X = !_X : X \to 1,$$
$$O_{k+1}^X = F_\Sigma(O_k^X) \circ \gamma.$$

The disclosure convention used in this paper starts with the visible top level rather than the empty terminal observation:

$$U_k^X = O_{k+1}^X, \qquad U_k(X, x) = U_k^X(x).$$

Thus U_0 contains the top-level Actor Graph with graph-valued continuations opaque, and U_{k+1} discloses one further frontier. This definition discloses the entire frontier simultaneously. It retains actor identity, accounts, transaction-history prefixes, event observations, link and transaction identities, potential topology, and process kind to the available depth; deeper graph-valued process positions are mapped to the terminal marker.

For human-readable display, define can $U_k(X, x)$ by sorting actor, link, and transaction records by $\prec_A, \prec_L, \prec_T$, and by sorting potential source/mediator/target triples lexicographically in the induced ActorID order. The first disclosed occurrence of an ActorID is rendered as its actor record; any repeated occurrence on the current disclosure path is rendered as ref(actor_id).

Lemma 1 (well-defined and faithful finite rendering). For every k, U_k^X is uniquely defined and independent of traversal order. Its canonical back-reference rendering is unique under the declared total identifier orders and determines the raw finite observation up to the unique identity-preserving isomorphism induced by those orders. On C0-well-formed states, the outer actor, link, transaction, and potential-topology list structure is already canonical, so equality of canonical renderings recovers literal equality of the corresponding positional polynomial shape; transaction-history order is preserved unchanged.

Proof. The map O_0^X is the unique map to the terminal object. If O_k^X is unique, then $F_\Sigma(O_k^X) \circ \gamma$ is unique by functoriality. Since $U_k^X = O_{k+1}^X$, every disclosure observation is unique. Canonical rendering sorts only the outer finite actor, link, transaction, and potential-topology carriers; it erases no constant coordinate and does not reorder transaction histories. Under C0 those outer carriers are already stored in the declared orders, so the sorting step is the identity on their positional list representation. First-occurrence rendering replaces repeated disclosed actor records only by back-references carrying the same stable ActorID; reversing that presentation step reconstructs the retained identity-labelled recursive positions uniquely. Hence the canonical rendering is faithful, and on C0-well-formed observations it recovers the same raw positional polynomial shape. □

Full eager `unfold` is therefore not required. **Disclosure** is depth-bounded, order-independent, account- and transaction-preserving, and identity-preserving.

6.7 Identity-preserving bisimulation

Definition 16 (identity-preserving Actor Graph bisimulation). A relation $\mathcal{B} \subseteq X \times Y$ between two F_Σ-coalgebras is an identity-preserving bisimulation when related states have matching F_Σ-observations under a relation lifting that preserves:

1. actor identities;
2. link identities and source/mediator/target incidence;
3. transaction identities, bound links, `source_actor_id`, `edge_actor_id`, and `target_actor_id` fields, and transaction data;
4. declared account-state observations;
5. transaction-history observations;
6. event observations;
7. local process kind and delegated ActorID;
8. potential source/mediator/target triples;
9. graph-valued process continuations, which must again be related by $\mathcal{B}$.

The largest such relation is denoted

$$x \sim_\iota y.$$

A domain may quotient account, transaction, event, or local-process labels by an explicitly declared observational equivalence. With exact label equality, the relation preserves the full finite observations specified above.

Account observations may be graded without changing the recursive machinery. **Structural account observation** preserves account identifiers, types, and conserved/non-conserved declarations. **Balance observation** additionally preserves visible typed balances. **Ledger observation** preserves canonical transaction identities, posting families, and their causal order. The selected grade is part of the declared observation equality; ledger observation is the account-side component of the ATG `(result, trace, ledger)` criterion.

Every F_Σ-coalgebra is finitely branching in the relevant sense: all recursive successor positions occur inside List_f. No additional finite-carrier assumption is made.

For the adequacy theorem, fix one account-observation grade

$$g \in \{\mathsf{structural}, \mathsf{balance}, \mathsf{ledger}\}$$

and use exact equality on every other visible actor, link, transaction, event, and process label. The symbols U_k, $\mathrm{can}\, U_k$, and $\sim_\iota$ in the theorem and proof are evaluated at this same fixed grade g.

Theorem 3 (finite-observation adequacy at a fixed observation grade). For C0-C7 well-formed Actor Graph coalgebras and states x, y,

$$\boxed{x \sim_\iota y \quad \Longleftrightarrow \quad \forall k \in \mathbb{N} : \ \mathrm{can}\, U_k(X, x) = \mathrm{can}\, U_k(Y, y)}$$

Equivalently, there exists a compatible family of identity-preserving isomorphisms between all finite observations.

Proof. For the forward direction, suppose $x \sim_\iota y$. Induction on k shows equality of the raw observations. At depth zero, the relation lifting preserves every constant coordinate of the outer F_Σ-shape. If corresponding recursive continuations are related, applying F_Σ to the induction hypothesis makes their depth-k observations equal, hence the depth-$k+1$ observations of x and y are equal. Canonical sorting then gives

$$\mathrm{can}\, U_k(X, x) = \mathrm{can}\, U_k(Y, y)$$

for every k.

For the reverse direction, define

$$R = \{(x, y) \mid \forall k \in \mathbb{N} : \ \mathrm{can}\, U_k(X, x) = \mathrm{can}\, U_k(Y, y)\}\,.$$

Fix $(x, y) \in R$. By C0 and Lemma 1, canonicalisation is faithful to the positional polynomial representation: the outer actor, link, transaction, and potential-topology lists are already stored in their declared canonical orders, while transaction-history order is retained as data. Hence equality of the canonical depth-zero renderings yields literal equality of the corresponding raw outer observations $U_0^X(x) = U_0^Y(y)$. This gives the same outer polynomial constructor, the same positional finite actor/link/transaction/potential-topology lists, and equality of every constant label. Their graph-valued process positions therefore have a unique common finite index set $\mathrm{Pos}(x, y)$. For each position p, let x_p and y_p be the corresponding continuations.

For every k, apply Lemma 1 again to the equality of the canonical depth-$k+1$ renderings. C0 converts that equality back into equality of the corresponding raw positional observations $U_{k+1}^X(x) = U_{k+1}^Y(y)$, which implies

$$\mathrm{can}\, U_k(X, x_p) = \mathrm{can}\, U_k(Y, y_p) \qquad \text{for every } p \in \mathrm{Pos}(x, y).$$

Hence $(x_p, y_p) \in R$ for every recursive position. All constant coordinates already agree, so

$$(\gamma_X(x), \gamma_Y(y)) \in \overline{F_\Sigma}(R),$$

where $\overline{F_\Sigma}$ is the standard polynomial relation lifting. Therefore R is an F_Σ-bisimulation, and $x \sim_\iota y$. □

Remark (where finite branching enters). The label sets Q, E, P, D may be infinite; they are constant coordinates and are compared directly. Finite branching is automatic because every recursive continuation occurs in a List_f position, so $\mathrm{Pos}(x, y)$ is finite at each step. Worrell's $\omega + \omega$ convergence bound concerns general finitary set functors [26]. The adequacy argument above uses the stronger finite-position polynomial structure of this specific functor and does not assume a finite coalgebra carrier.

Identity-preserving bisimulation lies strictly between literal equality of stored specifications and structural bisimulation that forgets identities:

$$= \subsetneq \sim_\iota \subsetneq \sim\,.$$

Different serialisations or back-reference placements may be $\sim_\iota$-equivalent without being literally equal. Two structurally identical graphs with different ActorID values may be bisimilar under $\sim$ but not under $\sim_\iota$.

6.8 Functoriality of finite disclosure

The finite observations above are not merely a family of state maps. They are compatible with every coalgebra morphism and with every truncation in the final sequence.

For $k \in \mathbb{N}$, put

$$Z_k = F_\Sigma^{k+1} 1, \qquad \mathbf{Obs}_k = \mathbf{Set}/Z_k.$$

Thus an object of $\mathbf{Obs}_k$ is a map into the depth-k observation carrier. For an F_Σ-coalgebra (X, γ), the disclosure map

$$U_k^X : X \longrightarrow Z_k$$

is such an object.

Theorem 4 (functorial finite-disclosure tower). For every k, the assignment

$$\mathcal{U}_k : \mathrm{Coalg}(F_\Sigma) \longrightarrow \mathbf{Obs}_k, \qquad (X,\gamma) \longmapsto U_k^X,$$

and

$$f : (X,\gamma) \to (Y,\delta) \longmapsto f : X \to Y$$

is a functor. Explicitly, every coalgebra morphism $f : (X,\gamma) \to (Y,\delta)$ satisfies the slice-morphism equation

$$U_k^Y \circ f = U_k^X.$$

If

$$r_k : Z_{k+1} \longrightarrow Z_k$$

is the connecting map of the final sequence and

$$R_k : \mathbf{Obs}_{k+1} \to \mathbf{Obs}_k$$

is postcomposition with r_k, then

$$\boxed{R_k \circ \mathcal{U}_{k+1} = \mathcal{U}_k}$$

strictly. Hence the raw finite disclosures form a compatible functorial tower.

Proof. Let $f : (X,\gamma) \to (Y,\delta)$ be a coalgebra morphism, so

$$\delta \circ f = F_\Sigma(f) \circ \gamma.$$

We prove

$$O_n^Y \circ f = O_n^X$$

by induction on n. For $n = 0$, both sides are the unique map $X \to 1$. If the equation holds at n, then

$$\begin{aligned} O_{n+1}^Y \circ f &= F_\Sigma(O_n^Y) \circ \delta \circ f \\ &= F_\Sigma(O_n^Y) \circ F_\Sigma(f) \circ \gamma \\ &= F_\Sigma(O_n^Y \circ f) \circ \gamma \\ &= F_\Sigma(O_n^X) \circ \gamma \\ &= O_{n+1}^X. \end{aligned}$$

Since $U_k = O_{k+1}$, the map f is a morphism in the slice $\mathbf{Set}/Z_k$. Identities and composition are inherited from **Set**, so $\mathcal{U}_k$ is a functor. The defining equations of the final sequence give

$$r_k \circ U_{k+1}^X = U_k^X$$

for every coalgebra, while R_k leaves the underlying arrow unchanged. Therefore $R_k \mathcal{U}_{k+1} = \mathcal{U}_k$. □

The theorem is stated for the raw observations. The human-readable serialisation can U_k is invariant under strict identity-preserving morphisms and equivariant under renamings that transport the declared identifier orders. No naturality claim is made for an arbitrary reordering followed by a literal string comparison.

6.9 Delegation cycles

A process may delegate to another ActorID, and delegation references may also be cyclic. Such a cycle is structurally valid. The base Actor Graph formalism does not silently assign it an execution meaning. An execution specialisation must choose and declare one of the following policies:

- reject unguarded delegation cycles as an execution error;
- require a guard, delay, event, transaction, or account-state change before recursive delegation;
- interpret the cycle through a domain-specific least or greatest fixed point.

This separates structural admissibility from execution safety.

6.10 Depth of disclosure

Not every local calculation should be disclosed as a separate actor. A modelling scale needs a stopping rule. Let M_{flat} keep a role inside a surrounding actor process and M_{open} disclose it as an actor or subgraph. Disclosure is justified when it changes an observable property required by the model: admissible actions, result, transaction attribution, causal trace, account consequences, rights, authority, risk, or evidence. The ATG specialisation gives a stricter ledger-sensitive version of this criterion.

7 Categorical identity-incidence-accountability skeleton

The full Actor Graph formalism combines three layers that should not be conflated:

1. finite sorted incidence and accountability data;
2. typed attributes, histories, policies, and well-formedness predicates;
3. recursive process positions interpreted coalgebraically.

This section gives a categorical theorem for the first layer and for fixed-type slices of part of the second. It does **not** identify the entire enriched and coinductive Actor Graph class with a presheaf topos.

7.1 Resolved disclosure views and ID-normal form

A disclosure view is **resolved** for the present construction when every included transaction is bound to an included realised link and every included posting names an included actor-account coordinate. Opaque external records remain valid Actor Graph data, but they are carried outside the resolved skeleton until their incidence or account target is disclosed.

For a resolved disclosure view D of an enriched Actor Graph $\mathcal{G}$, write $A|_D$, $\Lambda|_D$, and $\Theta|_D$ for the actor, link, and transaction carriers visible in the view. Define

$$A_D = \iota[A|_D], \qquad L_D = \ell[\Lambda|_D], \qquad T_D = \theta[\Theta|_D].$$

Let the globally attributable account-coordinate set be

$$Q_D = \bigsqcup_{a \in A|_D} \{(\iota(a), q) \mid q \in \mathrm{dom}(\mathrm{accounts}(a))\}.$$

Represent the finite posting family of each transaction as a map

$$p_\tau : J_\tau \to \mathsf{Posting}$$

from a finite occurrence set J_τ. Define

$$P_D = \bigsqcup_{\tau \in \Theta|_D} (\{\theta(\tau)\} \times J_\tau).$$

For $j \in J_\tau$, write $p_{\tau,j} = p_\tau(j)$. Tagging by transaction identity and posting occurrence preserves

duplicate posting values as distinct elements without imposing an arbitrary global order.

Definition 17 (resolved Actor Graph accountability skeleton). The resolved accountability skeleton of $(\mathcal{G}, D)$ is

$$K_D(\mathcal{G}) = (A_D, L_D, T_D, Q_D, P_D; s_D, m_D, t_D, b_D, o_D, \pi_T, \pi_Q),$$

where

$$\begin{aligned} s_D(\ell(\lambda)) &= \iota(s(\lambda)), \\ m_D(\ell(\lambda)) &= \iota(\varepsilon(\lambda)), \\ t_D(\ell(\lambda)) &= \iota(t(\lambda)), \\ b_D(\theta(\tau)) &= \mathrm{link_id}(\tau), \\ o_D(\alpha, q) &= \alpha, \\ \pi_T(\theta(\tau), j) &= \theta(\tau), \\ \pi_Q(\theta(\tau), j) &= (\mathrm{actor_id}(p_{\tau,j}), \mathrm{account_id}(p_{\tau,j})). \end{aligned}$$

Within the normalised skeleton, the source, mediator, and target identities of a transaction are derived rather than duplicated:

$$s_D \circ b_D, \qquad m_D \circ b_D, \qquad t_D \circ b_D.$$

Likewise, the owner of a posting target is $o_D \circ \pi_Q$.

Lemma 2 (ID-normal-form lemma). Every resolved disclosure view of an enriched Actor Graph is isomorphic, on its actor/link/transaction/account/posting incidence data, to its accountability skeleton $K_D(\mathcal{G})$. The normal form is unique up to sort-preserving isomorphism.

Proof. The maps ι, ℓ, and θ are injective, hence each is a bijection from the disclosed carrier onto its image. Transport s, t, ε, transaction binding, account ownership, and posting attribution along these bijections. The displayed equations define the transported maps uniquely. Account coordinates are already globally attributable after pairing the local coordinate with ActorID, and posting occurrences are made distinct by the tagged disjoint union. The inverse transport reconstructs the original resolved incidence data. Any two such transports differ only by sort-preserving bijections commuting with the structure maps. □

7.2 The Actor Graph accountability schema

Definition 18 (Actor Graph accountability schema). Let $\mathbb{S}_{\mathrm{AG}}$ be the free small category generated by the objects

$$\mathsf{A}, \mathsf{L}, \mathsf{T}, \mathsf{Q}, \mathsf{P}$$

and arrows

$$\begin{gathered} s, m, t : \mathsf{L} \to \mathsf{A}, \qquad b : \mathsf{T} \to \mathsf{L}, \\ o : \mathsf{Q} \to \mathsf{A}, \qquad \pi_T : \mathsf{P} \to \mathsf{T}, \qquad \pi_Q : \mathsf{P} \to \mathsf{Q}. \end{gathered}$$

An **Actor Graph accountability instance** is a functor

$$X : \mathbb{S}_{\mathrm{AG}} \to \mathbf{Set}.$$

The five carrier sets represent the ActorID, LinkID, and TransactionID sorts, attributable account coordinates, and posting occurrences. The arrows represent source/mediator/target incidence, mandatory transaction-to-link binding, account ownership, and posting attribution.

Definition 19 (skeleton morphisms and identity-faithful embeddings). A skeleton morphism $f : X \to Y$ is a natural transformation. Equivalently, it is a family

$$(f_A, f_L, f_T, f_Q, f_P)$$

that commutes with all seven generating maps. A **strict identity embedding** is a monomorphism whose components on $\mathsf{A}, \mathsf{L}, \mathsf{T}$ are literal inclusions of the relevant identifier sets. A **renaming embedding** is a monomorphism whose five components are injective; its first three components are sort-preserving renamings of ActorID, LinkID, and TransactionID, jointly compatible with source/mediator/target incidence, transaction binding, account ownership, and posting attribution as naturality requires.

Write

$$\mathbf{AGSkel} = [\mathbb{S}_{\mathrm{AG}}, \mathbf{Set}]$$

for the ambient category of accountability skeletons and structure-preserving morphisms. Its wide subcategory of monomorphisms contains the strict and renaming embeddings used for namespace-safe inclusion and gluing.

Theorem 5 (presheaf realisation of the Actor Graph accountability skeleton). The category of Actor Graph accountability-schema instances and their structure-preserving morphisms is

$$\mathbf{AGSkel} = [\mathbb{S}_{\mathrm{AG}}, \mathbf{Set}] \cong \widehat{\mathbb{S}_{\mathrm{AG}}^{\mathrm{op}}}.$$

Consequently:

1. **AGSkel** is complete and cocomplete;
2. limits and colimits are computed sortwise;
3. a morphism is monic exactly when all five components are injective;
4. **AGSkel** is a presheaf topos and therefore adhesive;
5. for any fixed type-data instance D_Σ, the slice $\mathbf{AGSkel}/D_\Sigma$ of typed skeletons is again a topos and hence adhesive.

Proof. Unpacking a functor $X : \mathbb{S}_{\mathrm{AG}} \to \mathbf{Set}$ gives exactly five sets and the seven displayed structure maps. Unpacking a natural transformation gives exactly five commuting component maps. These constructions are mutually inverse on objects and arrows. Definition 17 and Lemma 2 send every resolved Actor Graph disclosure view to such an instance. Conversely, every finite instance admits an enriched realisation with pairwise tagged identity sorts, null actor processes, realised potential triples, the prescribed transaction bindings and account ownership, one trivial non-conserved account type, zero-valued posting effects, and finite actor histories containing the relevant transaction identities in any declared local order. Thus the schema introduces no additional finite incidence shapes beyond the Actor Graph skeleton. The full Set-valued category is retained because restricting to finite instances alone would lose general categorical completeness.

A covariant functor category $[\mathbb{S}_{\mathrm{AG}}, \mathbf{Set}]$ is the presheaf category on $\mathbb{S}_{\mathrm{AG}}^{\mathrm{op}}$. Standard presheaf-topos results give pointwise limits and colimits and pointwise characterisation of monomorphisms [49]. Toposes are adhesive, so pushouts along monomorphisms are Van Kampen and stable under pullback [50, 51]. Slices of a topos are toposes, yielding the typed statement. □

Remark 3 (ambient infinite instances). The full Set-valued presheaf category includes infinite instances as categorical closure. The theorem does not claim that every such infinite instance is a resolved disclosure view. The converse realisation argument above is asserted for finite instances; infinite instances require separately declared resolution and admissibility conditions.

Corollary 2 (namespace-safe structural gluing). Let

$$K_1 \xleftarrow{i_1} J \xrightarrow{i_2} K_2$$

be a span of skeleton monomorphisms. Assume identifiers outside the common interface have been injectively renamed so that their actor, link, transaction, account-coordinate, and posting-occurrence carriers are disjoint. Then the pushout

$$K_1 +_J K_2$$

exists and is computed by sortwise disjoint union modulo the explicitly shared interface. The induced structure maps preserve source/mediator/target incidence, transaction binding, account ownership, and posting attribution.

Proof. Theorem 5 gives pointwise pushouts. The namespace condition makes every carrier pushout an ordinary disjoint union with only the interface images identified. Naturality of i_1 and i_2 ensures that the two definitions of every structure map agree on the interface, so the universal property induces a unique total map on each pushout carrier. □

The corollary proves **structural gluing**, not yet full operational composition. Event order, canonical-history reconciliation, registry coherence, graph-valued processes, potential topology, authority, balance policy, and account-coherent aggregation remain decorations or admissibility judgments that must be shown stable under a chosen enriched composition.

7.3 Categorical non-reconstructability

Theorem 1 can now be strengthened from recoverability of individual representations to an obstruction against functorial reconstruction.

Theorem 6 (no functorial reconstruction after identity collapse). Let

$$Q : \mathbf{AGSkel} \to \mathcal{D}$$

be a representation functor. If there are non-isomorphic skeletons G, H with

$$QG \cong QH,$$

then no functor

$$R : \mathcal{D} \to \mathbf{AGSkel}$$

can satisfy

$$R \circ Q \cong \mathrm{Id}_{\mathbf{AGSkel}}.$$

In particular, any representation functor that collapses one of the three witness pairs used in Theorem 1—parallel links sharing one mediator, repeated transactions through one link, or distinct transactions sharing one mediator—admits no functorial left inverse on the full admitted class.

Proof. Suppose such an R and natural isomorphism $RQ \cong \mathrm{Id}$ existed. Then

$$G \cong RQG \cong RQH \cong H,$$

where the middle isomorphism is R applied to $QG \cong QH$. This contradicts $G \ncong H$. The witness pairs of Theorem 1 are non-isomorphic precisely because they differ in the cardinality or incidence of the relevant LinkID or TransactionID sort, while the hypothesised collapsed representation identifies them. □

The proof uses no property of R beyond the fact that every functor preserves isomorphisms. The obstruction therefore applies to every functorial decoder, irrespective of any additional algebraic, logical, or operational structure carried by $\mathcal{D}$.

Remark 4 (standard machinery and Actor-Graph-specific content). The general facts about functor categories, presheaf toposes, and adhesivity are standard [49–51]. The Actor-Graph-specific contribution is the schema $\mathbb{S}_{\mathrm{AG}}$, the ID-normalisation of the triple-identity accountability data into that schema, the identification of the exact monomorphisms that preserve namespace and attribution, and the reconstruction obstruction induced by the non-collapse witnesses. No new general-purpose construction in category theory is claimed.

8 Structural fractality

Recursion and fractality are related but not identical. **Recursion** is the existence of graph-valued actor processes. **Cyclic recursion** is a return, directly or indirectly, to an already encountered ActorID. **Structural fractality** is the invariant that every disclosed graph-valued process is again governed by the same Actor Graph signature, including the account and transaction-bearing actor contract.

Definition 20 (structural fractal closure). A class $\mathcal{C}$ of graph specifications has structural fractal closure over signature Σ when, for every $G \in \mathcal{C}$ and every actor a with graph-valued process process$(a) = H$:

1. $H \in \mathcal{C}$;
2. source, mediator, and target roles in H are actors of the same universal type;
3. actor, link, and transaction identities retain their meanings across the disclosure boundary;
4. every disclosed actor again carries identity, accounts, transactions, events, and process;
5. further graph-valued processes remain admissible;
6. cyclic return to an already encountered ActorID is represented as a stable reference, not as a fresh copy.

Proposition 5 (Actor Graph fractal closure). The class $\mathfrak{AG}_\Sigma$ has structural fractal closure.

Proof. By the Lambek lemma, the structure map of the final coalgebra is an isomorphism

$$\zeta : \nu F_\Sigma \cong F_\Sigma(\nu F_\Sigma).$$

Every recursive process position of F_Σ therefore contains another member of the same final carrier. The invariant well-formed subcoalgebra $\mathfrak{AG}_\Sigma$ preserves the same identity, account, transaction, event, process, link, and potential-topology constraints at each such position. □

Thus the constructor reappears under observation at every finite depth:

$$U_k(G) \text{ is an Actor Graph observation over } \Sigma \quad \text{for every } k.$$

The informal formula

$$AG = AG(AG)$$

is shorthand for coinductive type closure and structural self-similarity. It is not literal set-theoretic equality.

8.1 Lambda-calculus reading of Actor Graph self-application

An earlier Simulator.Company technical note expressed the same intuition in the self-application form [40]

$$AG = (\lambda AG.\, AG(AG))(\lambda AG.\, AG(AG)).$$

Alpha-renaming the bound variable exposes the standard untyped lambda-calculus term

$$\Omega = (\lambda x.\, x\, x)(\lambda x.\, x\, x), \qquad \Omega \longrightarrow_\beta \Omega.$$

Read literally, Ω has no normal form: it denotes unguarded self-application and reproduces itself under beta reduction [37, 38]. It is therefore useful as an engineering mnemonic for **closure under self-application**, but it is not by itself a productive execution semantics for an Actor Graph.

A recursive construction with an explicitly declared graph transformer is better represented by a fixed point $G = \Phi(G)$:

$$G = \Phi(G), \qquad G = Y\Phi, \qquad Y\Phi =_\beta \Phi(Y\Phi),$$

where Y is Curry's fixed-point combinator and $=_\beta$ denotes beta-convertibility. The lambda-calculus equation says that the recursively defined graph is beta-convertible to one application of its constructor. Productive execution additionally requires a guarded discipline—for example a delay, event, transaction, state transition, or another condition that prevents immediate unguarded self-reduction—and does not follow from Y alone. The coinductive formulation of this paper then moves from term reduction to persistent system observation:

$$\nu F_\Sigma \cong F_\Sigma(\nu F_\Sigma), \qquad \mathfrak{AG}_\Sigma \subseteq \nu F_\Sigma.$$

The three notations therefore serve different purposes:

- Ω captures unrestricted self-application;
- $Y\Phi$ captures a declared recursive fixed point, productive when the execution discipline is guarded;
- νF_Σ captures a possibly cyclic Actor Graph as a persistently identifiable system observed through the finite views U_k.

The formal Actor Graph semantics in this paper is the third. It admits the intuition encoded by the earlier lambda notation without identifying self-computation with non-terminating eager reduction.

8.2 Edge fractality

Fractality applies to relations as well as to vertex-role actors. Since every edge actor $e = \varepsilon(\lambda)$ has the same contract,

$$\mathcal{C}(e) = \langle \iota(e), \text{accounts}(e), \text{transactions}(e), \text{events}(e), \text{process}(e) \rangle,$$

it may satisfy

$$\text{process}(e) = G_e.$$

The relation visible as one edge at disclosure scale k may therefore reveal a complete Actor Graph at scale $k+1$. Transactions executed through the outer relation remain attributable to $\iota(e)$, while disclosed internal actors and links explain their processing and postings. Edge actors inside G_e may disclose further graphs or return cyclically to $\iota(e)$.

When outer accounts of e are declared summaries of internal accounts, Definition 9 requires the declared aggregation to commute with disclosure. For conserved types, each internal transaction remains balanced under its declared boundary. Thus the accountable grammar recurs together with the graph grammar: an accountable relation disclosed is a graph of accountable relations. Figure 10 makes the full invariant explicit, including accounts, transactions, events, and process.

8.3 Disclosure notation and instance-level witnesses

Write

$$a \rhd G_a$$

to mean one disclosure step of the graph-valued process of actor a. A particular Actor Graph can be flat, acyclically recursive, or cyclically recursive. Non-trivial instance-level fractality is witnessed by either a positive-depth chain

$$a_0 \rhd G_{a_0} \ni a_1 \rhd G_{a_1} \ni \cdots,$$

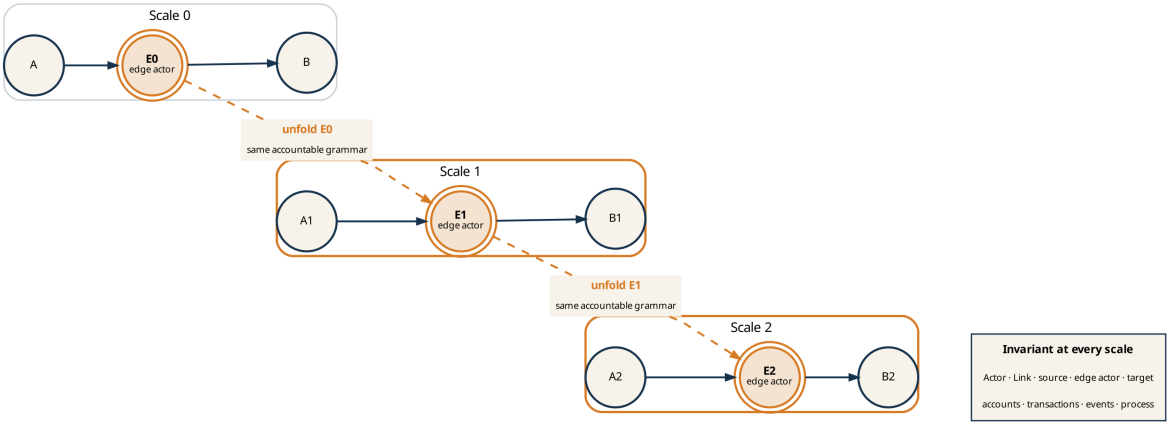


Figure 10: Structural fractality. The full Actor Graph grammar recurs when an actor-valued edge is disclosed: actor, link, source/mediator/target roles, accounts, transactions, events, and process.

or a recursive strongly connected component in $\prec_R$.

A finite cyclic specification may generate unbounded disclosure depth even though the stored representation and every finite observation are finite.

8.4 Structural, not geometric, self-similarity

No metric scaling ratio, exact geometric similarity, Hausdorff dimension, or infinite visual repetition is assumed. Actor Graph fractality is **typed, topological, operational, transactional, and identity-preserving**. The number of actors, topology, account values, transaction histories, events, processes, and physical scale may differ at every level. What recurs is the grammar of acting entities and acting relations.

9 Layers and graph types

The patent depicts empty graphs, trees, two-dimensional cyclic graphs, three-dimensional graphs, three-dimensional trees, and multilayer constructions [1]. These labels should not be confused with new combinatorial graph classes unless geometric coordinates are explicitly included.

Definition 21 (layered Actor Graph view). Let $\mathcal{L}$ be a set of layer labels and

$$\rho : A \to \mathcal{L}$$

a layer-assignment map. A layer $L \in \mathcal{L}$ induces the subgraph on actors with $\rho(a) = L$, together with any selected incident links. Links may remain within a layer or connect actors in different layers.

A **two-dimensional view** is a drawing or embedding of a selected Actor Graph view into one plane. A **three-dimensional view** assigns actors or layers coordinates in three-dimensional display space. A **3D tree** is a tree topology shown using such an embedding. These are visual or geometric representations of the same Actor Graph topology unless coordinate data are added to the formal signature. Figure 11 shows the corresponding ordinary vertex projections; edge actors and link nodes are intentionally suppressed.

Layering is orthogonal to coinductive recursion. A graph actor may reference a multilayer graph, actors in one layer may be edge actors in another view, and recursive disclosure may cross layer or jurisdiction boundaries.

A sequence

$$G_0 \subseteq G_1 \subseteq G_2 \subseteq \cdots$$

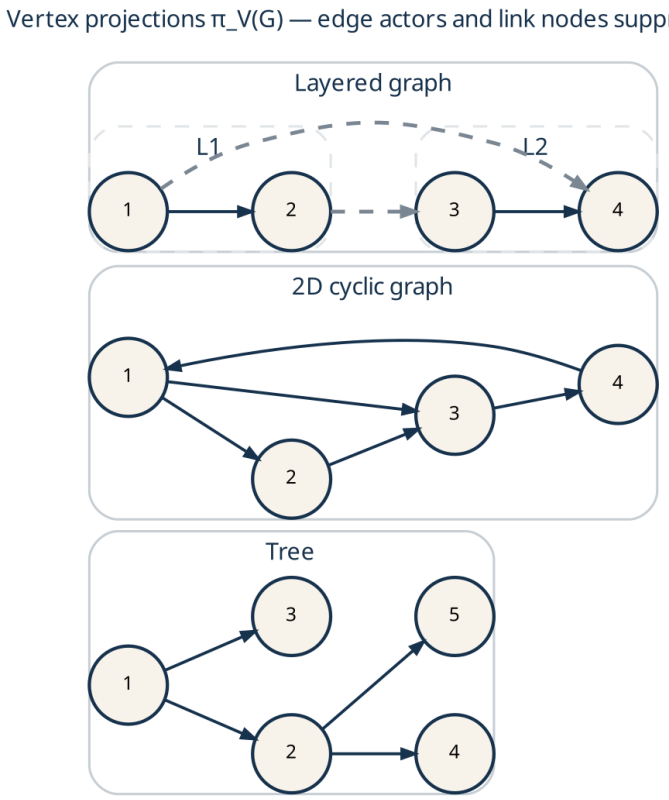


Figure 11: Examples of the ordinary vertex projection π_V(G): a tree, a two-dimensional cyclic view, and a layered view. Edge actors and link nodes are suppressed; dashed grey lines denote cross-layer relations.

may represent incremental construction or increasing disclosure. It is distinct from the recursive observation sequence $U_k(G)$: the former adds structure to a model, while the latter reveals structure already referenced by the model.

10 Matrix representations

Matrices are analytical representations of Actor Graph topology; they are not the definition of the object. All matrices in this section are computed for one declared **disclosure view** D. A disclosure view fixes the actors treated as vertex-role actors, the realised links visible at that level, and the treatment of boundary graph references. This prevents path counts from mixing several disclosure levels or counting the same stable actor identity twice merely because it appears in folded and disclosed views.

Let

$$V_D(G) = \{a_1, \dots, a_n\}$$

be the vertex-role actors in the view and

$$\Lambda_D = \{\lambda_1, \dots, \lambda_m\}$$

the visible realised links. The subscript D is suppressed below. Write

$$s_r = s(\lambda_r), \quad t_r = t(\lambda_r), \quad e_r = \varepsilon(\lambda_r), \quad \eta_r = \iota(e_r).$$

For a binary matrix B, the notation $B^{[k]}$ denotes the k-th power over the Boolean semiring:

$$B^{[0]} = I, \qquad B^{[k+1]} = B^{[k]} \odot B,$$

where multiplication uses conjunction and addition uses disjunction. Ordinary numeric powers retain the notation B^k.

10.1 Vector-valued adjacency matrix

Let ActorID* be the set of finite vectors over ActorID. Links are ordered by the strict total order $\prec_L$ on their `link_id` values.

Definition 22 (Actor Graph adjacency matrix). The adjacency matrix is

$$\mathbf{A}^{AG} = (\mathbf{a}_{ij})_{n\times n} \in (\text{ActorID}^*)^{n\times n},$$

where

$$\boxed{\mathbf{a}_{ij} = \left[\eta_r \mid s_r = a_i,\ t_r = a_j\right]_{\prec_L}}$$

is the ordered vector of `actor_id` values of all edge actors directed from a_i to a_j.

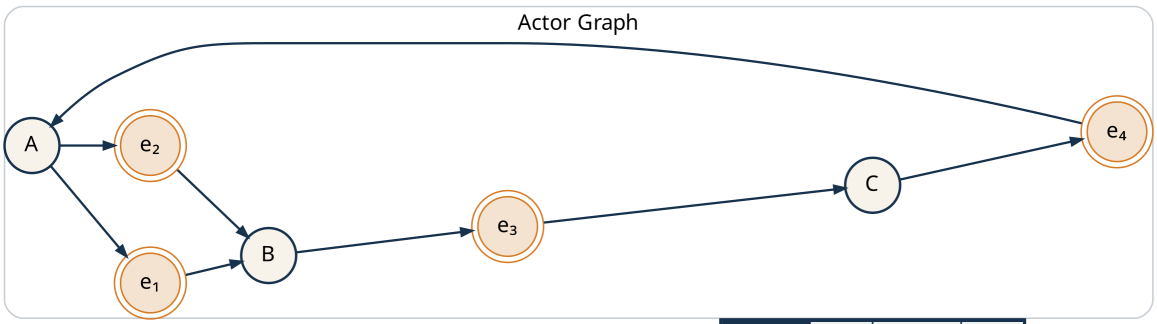
If no realised link exists,

$$\mathbf{a}_{ij} = [\,].$$

If several links connect the pair,

$$\mathbf{a}_{ij} = [\eta_{r_1}, \eta_{r_2}, \ldots, \eta_{r_k}].$$

The cell stores identities, not copies of state. Accounts, events, process, rights, graph references, and current enablement are resolved through the relevant edge-actor identity. If the same edge actor performs several distinct links between the same pair, its ActorID may occur more than once; the ordering and companion incidence labels preserve the separate `link_id` occurrences. Figure 12 shows the correspondence between the identity-preserving scheme and its vector-valued adjacency matrix.



A^AG	A	B	C
A	[ ]	[e1,e2]	[ ]
B	[ ]	[ ]	[e3]
C	[e4]	[ ]	[ ]

Figure 12: Correspondence between an Actor Graph scheme and its vector-valued adjacency matrix.

10.2 Semiring path interpretation

A vector is the primary storage representation requested by the Actor Graph definition. For algebraic path composition, it can be mapped into an established semiring construction from weighted-automata theory [21].

Let

$$\Gamma = \mathsf{LinkID} \times \mathsf{ActorID}$$

and let

$$K_\Gamma = (\mathcal{P}_{\text{fin}}(\Gamma^*), \cup, \cdot, \emptyset, \{\epsilon\})$$

be the finite-language semiring, where multiplication is language concatenation. Define

$$\widehat{\mathbf{A}}_{ij} = \left\{ [(\ell(\lambda_r), \eta_r)] \;\middle|\; s_r = a_i,\ t_r = a_j \right\}.$$

Then $\widehat{\mathbf{A}}_{ij}^k$ contains link/edge-actor label sequences along length-k walks in the fixed disclosure view. If multiplicities of identical words must be retained, use the noncommutative formal-power-series semiring $\mathbb{N}\langle\langle\Gamma^*\rangle\rangle$, or an equivalent multiset presentation.

This is not claimed as a new matrix algebra. Semiring matrices and weighted automata already provide labelled-path composition [21]. What they do **not** supply by themselves is composition of edge-actor processes, account effects, policy, authority, state-dependent enablement, compensation, or recursive graph disclosure. A process-aware enrichment of this path algebra remains open.

10.3 Binary adjacency projection

Define

$$a_{ij}^{(0)} = \begin{cases} 1, & \mathbf{a}_{ij} \neq [\,], \\ 0, & \mathbf{a}_{ij} = [\,]. \end{cases}$$

The matrix $\mathbf{A}^{(0)} = (a_{ij}^{(0)})$ records only the existence of at least one edge actor.

10.4 Multiplicity matrix

The multiplicity matrix is

$$\mathbf{M} = (m_{ij}), \qquad m_{ij} = |\mathbf{a}_{ij}|.$$

It records the number of realised links between each ordered pair.

10.5 Source and target incidence matrices

The source incidence matrix is

$$\mathbf{B}^- = (b_{ir}^-) \in \{0,1\}^{n \times m}, \qquad b_{ir}^- = 1 \iff s_r = a_i,$$

and the target incidence matrix is

$$\mathbf{B}^+ = (b_{ir}^+) \in \{0,1\}^{n \times m}, \qquad b_{ir}^+ = 1 \iff t_r = a_i.$$

For transaction analysis it is also useful to enumerate all visible actors, including actors occupying only the edge role, as

$$\bar{A}_D = V_D(G) \cup E_A(G) = \{\bar{a}_1, \dots, \bar{a}_{\bar{n}}\}.$$

Let $\bar{\mathbf{B}}^-$ and $\bar{\mathbf{B}}^+$ be the zero-extended source and target incidence matrices on $\bar{A}_D$, and define mediator incidence

$$\mathbf{B}^e = (b^e_{ir}) \in \{0,1\}^{\bar{n}\times m}, \qquad b^e_{ir} = 1 \iff e_r = \bar{a}_i.$$

Each column r is labelled by

$$(\ell(\lambda_r), \eta_r),$$

namely the link identity and the identity of its edge actor. The signed incidence matrix is

$$\mathbf{B} = \mathbf{B}^+ - \mathbf{B}^-.$$

For self-links the signed column vanishes, so $\mathbf{B}^-$ and $\mathbf{B}^+$ must be retained when self-links matter.

10.6 Relationship between adjacency and incidence

The multiplicity matrix is

$$\mathbf{M} = \mathbf{B}^-(\mathbf{B}^+)^{\mathsf{T}}.$$

The primary matrix is reconstructed by restoring edge-actor labels:

$$\mathbf{a}_{ij} = \left[\eta_r \;\middle|\; b^-_{ir} b^+_{jr} = 1\right]_{\prec_L}.$$

Proposition 6 (reconstruction). The pair $(\mathbf{B}^-, \mathbf{B}^+)$, together with the ordered column labels $((\ell(\lambda_r), \eta_r))_{r=1}^{m}$, uniquely determines $\mathbf{A}^{AG}$.

Proof. For each ordered pair (i, j), select the columns whose source and target indicators are both one, then place the corresponding η_r values in increasing `link_id` order. □

10.7 Loss of identity under numeric projection

Proposition 7 (non-injectivity of numeric adjacency). The maps

$$\mathbf{A}^{AG} \mapsto \mathbf{A}^{(0)} \qquad \text{and} \qquad \mathbf{A}^{AG} \mapsto \mathbf{M}$$

are not injective.

Proof. Consider two Actor Graphs with the same two vertex actors and one link from the first to the second. In one graph the edge actor has identity η_p and a payment process; in the other it has identity $\eta_c \neq \eta_p$ and a credit-decision process. Both produce the same binary and multiplicity matrices but different primary cells, $[\eta_p]$ and $[\eta_c]$. □

The result remains true when edge-actor identities coincide but link identities, histories, or graph contexts differ.

10.8 Degree matrices

The out-degree and in-degree, counting parallel links in the selected disclosure view, are

$$d_i^{\text{out}} = \sum_j m_{ij}, \qquad d_i^{\text{in}} = \sum_j m_{ji}.$$

The degree matrices are

$$\mathbf{D}_{\text{out}} = \operatorname{diag}(\mathbf{M}\mathbf{1}), \qquad \mathbf{D}_{\text{in}} = \operatorname{diag}(\mathbf{M}^{\mathsf{T}}\mathbf{1}).$$

These values describe topology, not execution volume.

10.9 Link-adjacency matrix

$$\mathbf{C} = (\mathbf{B}^{+})^{\mathsf{T}}\mathbf{B}^{-}$$

has links as rows and columns. Its entry is one exactly when the target actor of one link is the source actor of the next link. Rows and columns retain `link_id` and edge-actor labels.

10.10 Transaction-adjacency matrix

Let the visible transaction occurrences in a selected execution interval or configuration history be

$$\Theta_D = \{\tau_1, \dots, \tau_q\}.$$

Transaction matrices are indexed by the all-actor carrier $\bar{A}_D$, so mediators that occur only in edge roles remain visible.

Definition 23 (transaction-adjacency matrix). The transaction-adjacency matrix is

$$\mathbf{A}^{\text{tx}} = (\mathbf{x}_{ij})_{\bar{n}\times\bar{n}},$$

where

$$\boxed{\mathbf{x}_{ij} = \left[\theta(\tau_r) \;\middle|\; \text{source_actor_id}(\tau_r) = \iota(\bar{a}_i), \text{target_actor_id}(\tau_r) = \iota(\bar{a}_j)\right]_{\prec_T}}$$

contains the transaction identities observed from $\bar{a}_i$ to $\bar{a}_j$. The canonical transaction record resolves the corresponding `link_id`, edge actor, `value_type`/`value`, protocol, and effects.

The numeric transaction-flow matrix is

$$f_{ij} = |\mathbf{x}_{ij}|.$$

Unlike $\mathbf{M}$, which counts structural links, $\mathbf{F} = (f_{ij})$ counts transaction occurrences.

10.11 Transaction incidence matrices

The transaction-adjacency matrix above may include transactions bound to explicitly opaque external link identities. Structural transport identities, however, require a locally realised link column. Define

$$\Theta_D^{\text{loc}} = \{\sigma_1, \dots, \sigma_{q_{\text{loc}}}\} \subseteq \Theta_D$$

to be the visible transaction occurrences whose `link_id` resolves to a locally realised link in Λ_D. Transactions bound to opaque external links remain represented in $\mathbf{A}^{\text{tx}}$, $\mathbf{F}$, and the posting matrices, but are excluded from the local structural-transport identities below. An execution profile may alternatively adjoin one explicit boundary-link column per opaque external `link_id`.

Define source, mediator, and target transaction-incidence matrices

$$\mathbf{J}^{-} = (j^{-}_{iu}), \qquad \mathbf{J}^{e} = (j^{e}_{iu}), \qquad \mathbf{J}^{+} = (j^{+}_{iu}) \in \{0,1\}^{\bar{n}\times q_{\text{loc}}}$$

by

$$j^{-}_{iu} = 1 \iff \text{source_actor_id}(\sigma_u) = \iota(\bar{a}_i),$$

$$j^{e}_{iu} = 1 \iff \text{edge_actor_id}(\sigma_u) = \iota(\bar{a}_i),$$
$$j^{+}_{iu} = 1 \iff \text{target_actor_id}(\sigma_u) = \iota(\bar{a}_i).$$

The local link-transaction incidence matrix

$$\mathbf{K}_{\text{loc}} = (k_{su}) \in \{0,1\}^{m \times q_{\text{loc}}}$$

is defined by

$$k_{su} = 1 \iff \text{link_id}(\sigma_u) = \ell(\lambda_s).$$

These matrices distinguish structural topology from the locally realised occurrences executed through it. Because every $\sigma_u \in \Theta^{\text{loc}}_D$ is bound to exactly one locally realised link, structural incidence transports to occurrence incidence:

$$\boxed{\mathbf{J}^{-} = \bar{\mathbf{B}}^{-}\mathbf{K}_{\text{loc}}, \qquad \mathbf{J}^{e} = \mathbf{B}^{e}\mathbf{K}_{\text{loc}}, \qquad \mathbf{J}^{+} = \bar{\mathbf{B}}^{+}\mathbf{K}_{\text{loc}}}$$

and the local numeric transaction-flow matrix satisfies

$$\boxed{\mathbf{F}_{\text{loc}} = \mathbf{J}^{-}(\mathbf{J}^{+})^{\mathsf{T}}}$$

in the selected all-actor disclosure view. If $\mathbf{F}_{\partial}$ counts transactions bound to opaque external links, then

$$\mathbf{F} = \mathbf{F}_{\text{loc}} + \mathbf{F}_{\partial}.$$

The displayed identities are the occurrence-layer analogues of $\mathbf{M} = \mathbf{B}^{-}(\mathbf{B}^{+})^{\mathsf{T}}$, with their domain of validity stated explicitly.

10.12 Account-posting matrices

Fix a numeric currency c and enumerate the relevant actor-account coordinates

$$\mathcal{Q}_c = \{q_1, \dots, q_h\}.$$

The account-posting matrix is

$$\mathbf{P}^{(c)}_{\text{acct}} = (p_{ur}) \in \mathbb{R}^{h \times q},$$

where p_{ur} is the signed effect of transaction τ_r on account coordinate q_u. Debit/credit may be retained as two non-negative matrices when the accounting domain requires separate sides.

For a transaction-selection vector $z \in \{0,1\}^q$, the numeric account update is

$$\Delta\mathbf{q}^{(c)} = \mathbf{P}^{(c)}_{\text{acct}} z.$$

For a closed double-entry boundary and conserved currency, every transaction column balances:

$$\mathbf{1}^{\mathsf{T}}\mathbf{P}^{(c)}_{\text{acct}} = \mathbf{0}^{\mathsf{T}}.$$

If value crosses the selected boundary, explicit external, settlement, or equity accounts are included before applying the conservation test. Non-numeric account domains require a typed effect algebra rather than ordinary real-valued matrix addition.

10.13 Paths, reachability, and distance

Within the declared disclosure view, $(\mathbf{M}^k)_{ij}$ counts length-k directed walks with multiplicity. The count does not cross graph-reference boundaries unless a different, more disclosed view is first constructed. It also does not imply current executability.

The Boolean reachability matrix is

$$\mathbf{R} = \mathbf{I} \vee \mathbf{A}^{(0)} \vee (\mathbf{A}^{(0)})^{[2]} \vee \cdots \vee (\mathbf{A}^{(0)})^{[n-1]},$$

and the directed distance matrix $\mathbf{\Delta} = (\delta_{ij})$ is defined by the least Boolean path length, with $\delta_{ij} = \infty$ when unreachable.

10.14 Laplacian projections

Because $\mathbf{A}^{AG}$ is vector-valued, a Laplacian is defined only after choosing a numeric projection:

$$\mathbf{L}_{\text{out}} = \mathbf{D}_{\text{out}} - \mathbf{M}, \qquad \mathbf{L}_{\text{in}} = \mathbf{D}_{\text{in}} - \mathbf{M}^{\mathsf{T}}.$$

A symmetrised Laplacian may be formed from $\mathbf{M} + \mathbf{M}^{\mathsf{T}}$. There is no unique projection-independent Actor Graph Laplacian.

10.15 Weighted matrices

Let each visible realised link have a numerical weight q_r, such as cost, latency, capacity, risk, probability, commission, or transaction count. The additive weighted projection is

$$\mathbf{W}^{(q)} = \mathbf{B}^{-} \operatorname{diag}(q_1, \dots, q_m)(\mathbf{B}^{+})^{\mathsf{T}}.$$

Non-additive quantities require an explicitly declared aggregation algebra.

10.16 Current-state adjacency

For execution configuration c, let $\text{enabled}(\lambda_r, c) \in \{0, 1\}$. Define

$$\mathbf{a}_{ij}(c) = \left[\eta_r \;\middle|\; s_r = a_i,\ t_r = a_j, \text{enabled}(\lambda_r, c) = 1\right]_{\prec_L}.$$

The structural matrix lists realised edge actors; the current-state matrix lists those currently admissible.

10.17 Potential adjacency

Because potential topology includes an admissible mediator, its primary representation is also vector-valued:

$$\mathbf{p}_{ij} = \left[\iota(e) \;\middle|\; (a_i, e, a_j) \in \text{mayLink}\right]_{\prec_A}.$$

The binary potential-adjacency projection is

$$p_{ij}^{(0)} = \begin{cases} 1, & \mathbf{p}_{ij} \neq [\,], \\ 0, & \mathbf{p}_{ij} = [\,]. \end{cases}$$

Thus $\mathbf{P}^{AG}$ describes which edge actors may connect each ordered actor pair, while $\mathbf{A}^{AG}$ describes which links are realised.

10.18 Block matrices for layers

For a layer partition, $\mathbf{A}^{AG}$ has block form. Diagonal blocks contain edge actors within a layer; off-diagonal blocks contain cross-layer edge actors. Every cell remains a vector of ActorID values for edge actors.

10.19 Recursive-reference matrix

Let

$$U = \{u_1, \dots, u_N\} \subseteq \mathsf{ActorID}$$

be the finite set of identities considered across a closed recursive specification or selected observation. Define

$$\mathbf{H}_R = (h_{pq}),$$
$$h_{pq} = 1 \iff u_q \in \mathsf{CarrierID}(G_{u_p}),$$

where G_{u_p} is the graph-valued process referenced by u_p, when one exists.

This is a recursive-reference matrix, not a containment-tree matrix. Direct self-reference appears as

$$h_{pp} = 1,$$

and indirect cyclic reference of length k as

$$(\mathbf{H}_R^{[k]})_{pp} = 1$$

in Boolean algebra. Strongly connected components of $\mathbf{H}_R$ identify recursive modules. Diagonal entries in its transitive closure are therefore meaningful cycle indicators, not defects.

10.20 Configuration-transition matrix

If an execution specialisation has finite configuration set $C_G = \{c_1, \dots, c_N\}$, define

$$t_{uv} = 1 \iff c_u \Longrightarrow c_v.$$

A stochastic specialisation may instead use transition probabilities. This matrix describes the state-transition system of the whole Actor Graph, not actor adjacency.

10.21 Matrix hierarchy

The matrix family is organised as follows:

1. **identity-preserving topology:** $\mathbf{A}^{AG}$;
2. **link-preserving topology:** $\mathbf{B}^{-}$, $\mathbf{B}^{+}$, and link/edge-actor labels;
3. **semiring-labelled paths:** $\widehat{\mathbf{A}}$;
4. **transaction occurrence:** $\mathbf{A}^{\mathrm{tx}}$, $\mathbf{J}^{-}$, $\mathbf{J}^{e}$, $\mathbf{J}^{+}$, and $\mathbf{K}$;
5. **account and ledger effects:** $\mathbf{P}^{(c)}_{\mathrm{acct}}$;
6. **numeric topology projections:** $\mathbf{M}$, $\mathbf{A}^{(0)}$, and degree matrices;
7. **analytic derivatives:** paths, reachability, distance, Laplacian, and weighted matrices;
8. **dynamic and recursive representations:** $\mathbf{A}^{AG}(c)$, $\mathbf{P}^{AG}$, $\mathbf{H}_R$, and $\mathbf{T}_G$.

Classical graph analysis is therefore available through projections, while the Actor Graph matrices preserve the identities of relation-performing actors, the identities of transaction occurrences, and the account effects attributed to them.

11 Worked examples

11.1 Transaction, accounts, and reversal

Consider three actors:

- c: customer;
- m: merchant;
- p: payment actor, occupying the edge role.

Let λ_p be the realised link

$$c \xrightarrow{p} m$$

with `link_id = pay-1`. The primary adjacency cell is

$$\mathbf{a}_{cm} = [p].$$

Adding a second realised link $c \to m$ for a loyalty-points leg of a split tender, performed by loyalty-programme actor w with `link_id = pts-1` and `value_type = Loyalty:PTS`, extends the same ordered-pair cell to

$$\mathbf{a}_{cm} = [p, w].$$

Each link keeps its own transaction occurrences, while the two mediators keep separate accounts and histories. A delivery relation running from merchant to customer would instead occupy $\mathbf{a}_{mc}$, not $\mathbf{a}_{cm}$: parallel links share one ordered pair.

A payment occurrence is

$$\tau_1 = \langle \texttt{tx-1001}, \texttt{pay-1}, c, p, m, \text{EUR}, 100, \\ \texttt{payment}, \text{preconditions}, \text{metadata}, \text{effects} \rangle.$$

Its transaction-adjacency cell is

$$\mathbf{x}_{cm} = [\texttt{tx-1001}].$$

Let the account coordinates be `Customer.cash`, `Merchant.cash`, and `Payment.fee`. The signed EUR posting matrix for this one transaction is

$$\mathbf{P}_{\text{acct}}^{(\text{EUR})} = \begin{pmatrix} -100 \\ +97 \\ +3 \end{pmatrix}.$$

The column balances:

$$-100 + 97 + 3 = 0.$$

Committing τ_1 appends `tx-1001` to the relevant histories, applies the postings, and emits transaction events. The edge actor p is not a passive label: its own fee account and transaction history change.

A reversal does not delete τ_1. It creates τ_2 with its own `transaction_id`, causal reference to τ_1, and compensating postings. The history therefore distinguishes the original payment and its reversal even when the final numeric balances return to their prior values.

Suppose p discloses an internal graph containing issuer, acquirer, payment network, anti-fraud, clearing, settlement, and reconciliation actors. The outer cell remains $[p]$; the outer transaction remains attributable to `edge_actor_id = p`; and the disclosed graph explains the internal transactions, events, and postings without replacing the identity of the payment relation.

11.2 Minimal cyclic Actor Graph

Let actors P and Q have identities p and q, and graph-valued processes such that

$$q \prec_R p, \qquad p \prec_R q.$$

The recursive-reference matrix is

$$\mathbf{H}_R = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Its Boolean square is

$$\mathbf{H}_R^{[2]} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix},$$

so both actors belong to one recursive strongly connected component.

The finite observations from P are:

$$U_0(P) = P[\text{process} = \text{opaque}],$$
$$U_1(P) = P[G_P(Q[\text{process} = \text{opaque}])],$$
$$U_2(P) = P[G_P(Q[G_Q(\text{ref}(p))])].$$

The observation remains finite because the repeated ActorID is rendered as a back-reference. Accounts and transaction-history observations associated with P and Q remain attached to those identities at every depth. Figure 13 combines the stored cyclic specification, the observations U_0 through U_2, and the recursive-reference matrix.

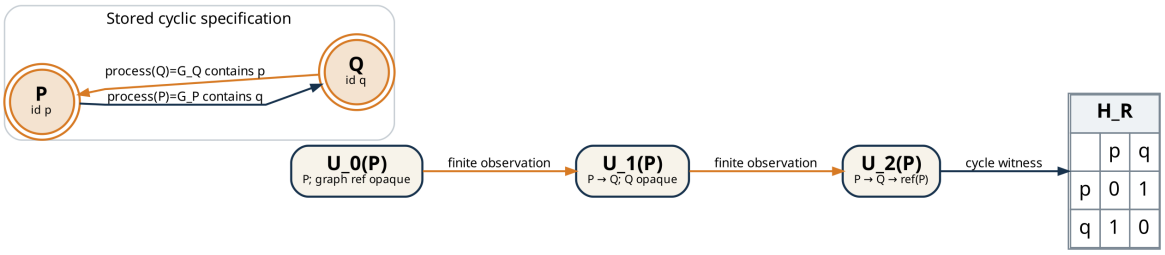


Figure 13: Minimal cyclic Actor Graph, its finite observations, and recursive-reference matrix.

12 Execution semantics

A static structural Actor Graph specifies identities and topology. An enriched Actor Graph additionally specifies accounts, transaction histories and records, events, processes, potential topology, and recursive references. A finite execution configuration may include:

- actor and edge-actor account states;
- append-only transaction histories;
- event histories;

- active triggers;
- graph and process versions;
- authority and policy conditions;
- enabled links and pending transactions.

Let $c \in C_G$ be a configuration. A recognised inter-actor interaction is written

$$c \xrightarrow{\tau} c',$$

where τ has a `transaction_id` and is bound to a realised link and its source, mediator, and target actors. In the canonical operational reading,

$$\text{RecognisedMessage}_{AG} \equiv \text{TransactionOccurrence}\,.$$

Local computation that has not yet crossed an actor boundary may remain inside a process. Once an interaction is recognised by the graph or changes attributable account state, it is transaction-identified.

A successful commit satisfies four coupled obligations:

1. **identity:** the transaction record is canonical and immutable;
2. **history:** the transaction identity is appended to the relevant actor histories;
3. **accounts:** all account changes are effects of that transaction;
4. **evidence:** transaction events record the causal transition.

The general AG definition does not force one concurrency or consensus semantics. Mailboxes, synchronous calls, asynchronous events, discrete-event simulation, Petri-style firing, probabilistic kernels, distributed commits, or institutional recognition protocols may implement the commit boundary. The Actor Graph fixes **what the persistent actors, links, accounts, and transaction occurrences are**; an execution specialisation fixes **how preconditions are checked, commits become authoritative, failures are handled, and concurrent transactions compose**.

Coinductive graph reference does not require infinite runtime expansion. An implementation may retain finite cyclic ActorID references, inspect $U_k(G)$ to a chosen depth, and execute only the locally enabled transaction process. Unguarded actor-delegation cycles must be rejected, guarded, or interpreted by an explicitly declared fixed-point policy.

For external actors, execution may mean recognition of an externally produced transaction or state claim rather than local control. This permits one Actor Graph to combine software execution with human, physical, legal, and institutional jurisdictions without pretending that all transitions are locally owned.

13 Active Transaction Graphs as a strict specialisation

Accounts and transactions are already present in the enriched Actor Graph. Active Transaction Graphs do not introduce them for the first time. They impose a stricter closure discipline on their use [2]:

$$\mathsf{ATG} \subseteq \mathfrak{AG}_{\Sigma}.$$

An ATG requires, at minimum:

1. every formally recognised inter-actor message is a transaction;
2. every transaction has a stable `transaction_id` and explicit source, edge, target, and link fields, together with protocol and effects;

3. account effects are attributable to transactions;
4. committed histories are append-only;
5. retries are idempotent or otherwise explicitly differentiated;
6. cancellation, reversal, and compensation are new transactions;
7. replay and provenance rules are declared;
8. execution is observed through result, trace, and ledger.

A representative transaction tuple is the record of Definition 5. Transactional actor memory remains

$$\text{TransactionalMemory}(a) = \text{Accounts}(a) + \text{ImmutableTransactionHistory}(a).$$

The observable semantics of an ATG execution is

$$\text{Sem}(M) = (\text{result}(M), \text{trace}(M), \text{ledger}(M)).$$

This gives a criterion for deciding whether an apparently simple relation may remain folded. Let M_{edge} retain a mediator as an edge actor and M_{flat} replace it with a passive relation. Even if results coincide, the models are not equivalent when trace or ledger differs:

$$D(M_{\text{edge}}, M_{\text{flat}}) = (\text{trace}(M_{\text{edge}}) \neq \text{trace}(M_{\text{flat}})) \vee (\text{ledger}(M_{\text{edge}}) \neq \text{ledger}(M_{\text{flat}})).$$
$$\text{result}(M_{\text{edge}}) = \text{result}(M_{\text{flat}}) \ \wedge \ D(M_{\text{edge}}, M_{\text{flat}}) \Longrightarrow \text{Sem}(M_{\text{edge}}) \neq \text{Sem}(M_{\text{flat}}).$$

The relation between the two levels can therefore be summarised as

$$\boxed{\text{ATG} = \text{AG}_{\text{accounts+transactions}} + \text{TransactionPrimacy} + \text{CommitInvariants} + \text{RTL}}$$

where $\text{RTL} = (\text{result}, \text{trace}, \text{ledger})$.

The general term **Actor Graph** should not be replaced by **Active Transaction Graph**: the latter is the stricter transactional semantics of the former.

14 Relation to adjacent formalisms

The central novelty boundary is not whether a relation can be reified, whether graphs can be hierarchical, or whether accounting can be expressed algebraically. Mature formalisms provide each of those capabilities. The Actor Graph claim concerns one native conjunction:

$$\boxed{\begin{array}{c}\text{structural link identity} + \text{transaction occurrence identity} \\ +\text{reusable mediator ActorID} + \text{uniform accountable process contract} \\ +\text{identity-preserving disclosure}\end{array}}$$

This section therefore acknowledges each closest neighbour and states only the remaining boundary.

14.1 Property graphs and RDF 1.2

Property graphs permit nodes and edges to carry labels and properties [14]. This supports state-like metadata on edges but normally preserves a node/edge type distinction. An edge does not automatically become a process-bearing actor that participates elsewhere without an explicit reification convention.

RDF reification and RDF 1.2 triple terms make statements about statements possible. RDF 1.2 permits a triple term to occur as an RDF term and permits a reifier to identify an occurrence separately from the triple term [15]. This is a genuine structural analogue of separating an occurrence from

reusable relation content. The Actor Graph distinction does not rest on denying that mechanism. It rests on the third identity: the mediator is an actor with its own accounts, transactions, events, and process, and graph-valued disclosure preserves that mediator identity and accountable continuity across scales.

14.2 Object-Role Modeling

Object-Role Modeling permits **objectification** or **nesting**: an instance of a fact type may be treated as an object about which further facts are stated [44]. ORM therefore supplies a direct precedent for turning a relationship occurrence into a role-playing object. Actor Graphs do not claim objectification as new. Their residual commitment is the conjunction of a persistent `link_id`, a distinct transaction occurrence, a reusable mediator ActorID with the full accountable process-bearing contract, and identity-preserving coinductive disclosure.

14.3 UML association classes and SysML association blocks

UML `AssociationClass` combines the properties of an association and a class: a concrete relationship instance can carry object-like attributes and participate in further modelling [52]. SysML association blocks and connector properties similarly permit a connector to be typed by an association class stereotyped as a block [53]. These are important industrial precedents for relation instances with identity and data. They do not by themselves supply the Actor Graph triple split. If one association-class instance is used simultaneously as the structural link and as the performer of that relation, then link identity and mediator identity are collapsed. A faithful UML or SysML encoding can avoid the loss by introducing a separate reusable mediator object, but that additional object is precisely the third discriminator required by Theorem 1. Actor Graph novelty therefore lies not in the encodability of object-like relations, but in making the link/occurrence/mediator separation and the accountable mediator contract native.

14.4 TypeDB and nested relations

TypeDB relations are first-class instances: they define roles, may own attributes, and may themselves play roles in other relations [16]. This is the closest industrial neighbour to first-class relation participation.

The distinction is precise. In Actor Graphs, one `link_id` identifies a concrete structural relation, one `transaction_id` identifies an occurrence through it, and a separate `edge_actor_id` identifies the reusable actor performing it. The mediator has the same operational contract as every other actor and may reference another Actor Graph while retaining identity. TypeDB demonstrates that relations can be first-class role players; it does not impose this three-identity, account-bearing, process-bearing, coinductively disclosed actor signature.

14.5 Reo and behavioural connectors

Reo is an exogenous coordination language in which complex connectors are compositionally built from channels with declared behaviour [41]. Constraint automata provide an operational semantics for such connectors, including stateful coordination, composition, equivalence, and refinement [42]. Reo is therefore the closest process-level neighbour on the axis of a mediator with its own behaviour.

The remaining boundary is not that Reo connectors are passive; they are not. Actor Graphs additionally treat the persistent relation's LinkID, the occurrence's TransactionID, and the mediator's ActorID as separate native sorts. The mediator owns accounts, transactions, events, and process under the same actor contract as its endpoints, and a graph-valued mediator process discloses while preserving that ActorID and accountable continuity. Reo connector composition and Actor Graph disclosure may be related by future translations, but neither formalism is reduced to the

other here.

14.6 Metagraphs, ubergraphs, and recursive hypergraphs

Metagraphs generalise graph edges to relationships among sets and support rich relational modelling [17]. Ubergraphs allow edges to contain other edges as elements [18]. Recursive multi-relational hypergraphs likewise permit hyperedges to act as vertices in other hyperedges [25]. These are direct precedents for recursive or higher-order relation structure.

Actor Graphs differ by assigning node, edge, and graph roles to one stable actor universe; separating structural relation, occurrence, and mediator identities; and requiring the mediator to carry accountable state, transaction history, events, and process. Recursion is expressed through stable actor references and identity-preserving bisimulation.

14.7 Higraphs and hierarchical graph transformation

Higraphs add explicit containment, orthogonality, and zooming to graph-based visual formalisms [19]. Hierarchical graph transformation develops rule-based transformation for graphs containing hierarchical structure [20]. These traditions are important precedents for multi-scale disclosure and transformation.

Actor Graphs do not claim hierarchy as new. Their additional commitment is that an actor occupying a vertex or edge role may itself reference a graph of the same actor-valued-edge signature while retaining one ActorID, including cyclic return and optional account-coherent aggregation.

14.8 Attributed graph transformation

Typed attributed graph transformation gives graphs node and edge data and develops mature double-pushout rewriting, concurrency, and consistency results over suitable adhesive high-level replacement categories [54]. Actor Graphs do not claim typed attributes, graph rewriting, or adhesivity as new. Their additional invariant is the recoverable separation of the persistent structural link, the transaction occurrence, and the reusable mediator, together with actor-owned accounts, transaction-attributed postings, and identity-preserving recursive disclosure. The presheaf skeleton of this paper is therefore a natural host for a future Actor-Graph-specific rewrite theory, but no preservation theorem for enriched Actor Graph rewrites is claimed here.

14.9 Semiring matrices and weighted automata

Weighted automata and formal power series use matrices over semirings to compose transition labels and weights along paths [21]. The finite-language semiring construction in the matrix section is therefore a bridge to established theory, not a novelty claim.

The remaining Actor Graph problem begins where ordinary semiring path composition stops: composing mediator processes, account effects, policy, authority, state-dependent enablement, compensation, and recursive disclosure of the edge actors named by path labels.

14.10 The REA accounting ontology and OeBTO

McCarthy's REA model organises economic phenomena around Resources, Events, and Agents [27]. Later REA work adds enterprise and policy-level specifications [28], Hruby develops REA-based business patterns for model-driven design [32], and ISO/IEC 15944-4 uses REA as the accounting and economic ontology of the Open-edi Business Transaction Ontology, including binary and mediated collaborations [29]. REA is therefore the closest ontological neighbour on the accountability axis.

The mapping is informative but not an identity. REA resources correspond broadly to economically relevant accountable state; REA events correspond to economic occurrences; REA agents

correspond to participating actors; duality connects give/get events. REA and OeBTO can represent third-party agents and mediated collaboration, so Actor Graph novelty cannot be based on claiming that mediation is absent there.

The remaining distinction is the native graph signature. Actor Graphs separate the persistent structural link, the transaction occurrence, and the reusable mediator into three identities. The mediator itself has the uniform actor contract and can occupy an edge role, own postings and balances, participate in other relations, and disclose into another Actor Graph while retaining identity. Account-coherent disclosure is a structural operation over the actor-valued edge; REA consolidation and economic-event modelling do not impose that particular recursive identity invariant. Actor Graphs may therefore host REA-compatible economic semantics, but they are not a renaming of REA.

14.11 DEMO and Enterprise Ontology

DEMO/Enterprise Ontology treats an organisation as a system of actor roles and transactions. Its transaction axiom distinguishes initiator and executor roles and organises coordination acts and facts together with production acts and facts into a recurring transaction pattern aimed at an agreed result [43]. It is therefore a direct neighbour on the axes of accountability, commitments, and institutional recognition of transitions.

Actor Graphs do not claim those organisational transaction patterns as new. Their additional structural commitment is that a transaction occurrence is bound to a persistent link performed by a separately identified reusable edge actor; that mediator may own postings and state, participate in other relations, and disclose coinductively without losing identity. A DEMO transaction pattern can therefore serve as an execution or institutional-recognition specialisation of an Actor Graph relation, while the general Actor Graph signature also covers non-DEMO computational, physical, and inter-organisational mediators.

14.12 Algebra of double-entry bookkeeping

Ellerman formalises double-entry bookkeeping through the group-of-differences construction and later extends the treatment to multidimensional value vectors [30, 31]. Definition 7 uses that established algebra for declared conserved quantitative types. The algebra itself is not an Actor Graph contribution.

The Actor Graph contribution is **placement and composition**: effects are attributed to stable ActorID values in vertex, edge, external, and graph roles; the structural link, occurrence, and mediator identities remain distinct; the mediator is an accountable party; balance is tested over all postings relative to an explicit boundary; and declared account aggregation may commute with identity-preserving disclosure. Double entry becomes one typed invariant of transactions in an actor-valued-edge graph rather than the definition of a separate ledger document.

14.13 Classical graph theory

Classical directed multigraphs provide vertices, links, source and target maps, adjacency and incidence matrices, paths, reachability, and Laplacians [3, 4]. Actor Graphs retain this structure under forgetful projection but add the actor-valued edge map, the three-identity split, the accountable process-bearing actor contract, potential topology, and coinductive graph references.

14.14 The Actor Model

The Actor Model treats actors as autonomous computational entities communicating through messages [5, 6]. Actor Graphs adopt stable acting entities and local behaviour but make the relation itself eligible to be an actor with its own accounts, transactions, events, and process. The model also extends beyond one runtime jurisdiction.

14.15 Petri nets

Petri nets distinguish places and transitions and provide strong enabling and concurrency semantics [7]. Actor Graphs do not deny those operational distinctions; they permit both roles to be represented by actors of one universal type with role-specific processes. ATGs provide a transactional embedding of finite Petri nets under stated restrictions [2].

14.16 Bigraphs and mobile systems

Bigraphs separate place structure from link structure [8], while Mobile Ambients model nested locations and mobility [9]. Actor Graphs use a different composition: place-like, node-like, edge-like, and graph-like roles retain actor identity and an operational contract.

14.17 Statecharts, DEVS, and Ptolemy II

Statecharts add hierarchy, orthogonality, and concurrency [10]. DEVS and Ptolemy II provide mature hierarchical and executable modelling frameworks [11, 12]. Ptolemy's `CompositeActor` is a particularly relevant precedent for recursive component composition. Actor Graphs add actor-valued edges, distinct link/transaction/mediator identities, one accountable actor contract across roles, external jurisdictions, and coinductive identity-preserving graph references.

14.18 Process mining

Process mining derives and checks process models against event logs [13]. Actor Graph events can supply such traces; ATGs additionally couple trace to result and ledger. Actor Graphs also model the mediators, transaction occurrences, and accountable states that produced the trace.

14.19 Presheaf and adhesive graph categories

Many graph-like data structures are represented by functor or presheaf categories, and adhesive categories were developed to make pushout-based graph gluing and rewriting well behaved [49, 50]. The fact that a presheaf category is a topos and that toposes are adhesive is established general machinery [49, 51].

The present categorical claim is therefore not priority for presheaves, toposes, adhesivity, or pushouts. It is the Actor-Graph-specific choice of five accountability sorts and seven attribution maps, the ID-normal-form correspondence from resolved disclosures to those instances, and the resulting no-reconstruction theorem for collapse of link, transaction, or mediator distinctions.

14.20 Comparative boundary

Formalism	First-class relation	Relation in relation	Separate relation / occurrence / mediator identities	Accountable process-bearing mediator	Identity-preserving graph disclosure
Property graph	Yes	By reification	No native triple split	No	No
RDF 1.2	Yes, as triple term/ reifier	Yes	Partly: reifier names an occurrence	No actor contract	No
ORM objectification	Yes, as objectified fact	Yes	Relationship occurrence can be objectified; no AG triple split	No uniform actor contract	No
TypeDB	Yes	Yes	No AG-style triple split	Attributes, but no required actor contract	No required AG unfolding
Reo	Connector/channel is first-class and behavioural	Connectors are compositional	No AG link / occurrence / mediator split	Stateful connector semantics; no actor account contract	Connector composition, not AG identity disclosure
Metagraph / ubergraph	Yes	Yes	No	No	Structural recursion only
Higraph / hierarchical graph	Partly	Via hierarchy	No	No	Hierarchical disclosure
DEMO / Enterprise Ontology	Organisational transaction pattern	Transactions compose into processes	Transaction and actor-role identities; no AG triple split	actor roles and institutional facts; no native edge-actor account contract	No AG coinductive disclosure
REA / OeBTO	Economic events and mediated collaborations	Through ontology relations	Event/agent distinctions, but no AG triple identity	Third-party agent possible; not a native edge-actor account contract	No AG identity-preserving disclosure
Double-entry / group of differences	No graph relation model	No	No	Ledger algebra only	No
Actor Model	Messages / channels	Not native	No	Actors only	Runtime-specific composition

Continued on next page

Formalism	First-class relation	Relation in relation	Separate relation / occurrence / mediator identities	Accountable process-bearing mediator	Identity-preserving graph disclosure
Actor Graph	**Yes, as actor**	**Yes**	**Yes: link / transaction / mediator**	**Yes: accounts / transactions / events / process**	**Yes, coinductively**

No row is presented as globally superior. The table isolates the exact conjunction for which the name **Actor Graph** is proposed.

14.20.1 Accounting-axis comparison

Formalism	Typed accountable state	Occurrence identity	Mediator owns accounts	Conserved-value algebra	Disclosure-compatible account continuity
REA / OeBTO	Yes, economically	Economic-event identity	Third-party agent may be modelled	Duality/economic semantics	Not an AG disclosure invariant
Double-entry algebra	Ledger coordinates	Journal-entry dependent	Not a graph concept	Yes	Not a graph concept
Actor Graph	**Yes, for every actor role**	**Explicit `transaction_id`**	**Yes, the edge actor is a posting owner**	**Typed; group of differences for conserved quantitative types**	**Declared account-coherent disclosure**

15 Actor Graphs and programming-language expressiveness

This section states an interpretive position and an open formal programme. It does not add a proved contribution to the definitions, propositions, and theorems above.

15.1 Computability is not the axis

Because `LocalProcess` may be implemented by a conventional programming language, Actor Graphs do not claim access to computations unavailable to a Turing-complete host. Computational equivalence is therefore conceded at the outset. Felleisen's framework addresses a different question: whether a construct of an extended language can be eliminated by a **local, syntax-directed translation** that leaves the pre-existing language constructs unchanged, or whether its elimination requires a transformation of the surrounding program [33]. In this sense, two languages may compute the same partial functions while differing materially in expressive power.

The same distinction applies here. An Actor Graph can be encoded in Java, C#, Python, a database schema, an RDF graph, or an ordinary labelled graph. The issue is not encodability. It is whether the native Actor Graph invariants can be supplied independently at each use site, or whether a faithful representation requires coordinated support across modules, persistent stores, queues, process runtimes, and observation tools.

$$\text{computational equivalence} \quad \text{does not imply} \quad \text{macro-expressibility equivalence}.$$

15.2 Lambda calculus as the local computational substrate

Lambda calculus isolates local computation into abstraction and application. Fixed-point combinators add recursion without changing the basic term language [37, 38]. Actor Graphs do not compete with this computational core. They provide an operational model around it.

Landin's correspondence between ALGOL 60 and Church's lambda notation demonstrated that a classical programming language can be described through a lambda-calculus semantic core [39]. The point here is the complementary direction: even when a program's local computation is captured by lambda abstraction and application, the operating system around that computation still requires persistent identities, recognised transactions, attributable accounts, causal evidence, authority, and relations that can themselves be disclosed as systems. Actor Graph therefore treats lambda-definable computation as an internal process semantics while retaining the larger operational model as the primary object.

Let Term_λ denote a selected lambda-calculus term language. An execution specialisation may interpret such terms through the process position of an actor:

$$\mathcal{J}_\lambda : \mathsf{Term}_\lambda \longrightarrow \mathsf{LocalProcess} \hookrightarrow \mathsf{Proc}_\Sigma(\mathfrak{AG}_\Sigma).$$

Lambda-definable computation therefore enters $\mathfrak{AG}_\Sigma$ through the process position of an actor, not as a subset of the final carrier.

This typing states a division of labour. Lambda calculus and the conventional languages compiled from or interpreted through equivalent computational mechanisms describe **how a local transformation is computed**. The Actor Graph describes **who performs it, through which persistent relation, in which recognised transaction, against which accounts, with which causal evidence, and at which recursively disclosed scale**. Pure lambda terms do not natively provide stable ActorID, LinkID, TransactionID, account ownership, institutional recognition, or an append-only causal history. All of these can be encoded with explicit state, effects, environments, stores, and protocols; the expressiveness question is whether that encoding remains local.

Conversely, Actor Graphs preserve classical programming rather than discarding it. A lambda term, function, service implementation, or compiled program can remain unchanged inside a `LocalProcess`. The graph supplies the identity-bearing and accountable system structure in which

that computation acts.

15.3 A Felleisen-style criterion for Actor Graph embeddings

Let L be a code-primary host language and let L_{AG} extend L with primitives for stable ActorID, LinkID, and TransactionID; actor-valued links; typed actor accounts; transaction commit; and identity-preserving finite disclosure. Let Obs_{AG} denote the declared observation consisting of the relevant topology, identities, transaction occurrences, account state, event evidence, and finite disclosure views.

Definition 24 (AG-local embedding). An embedding

$$T : L_{AG} \longrightarrow L$$

is **AG-local** when:

1. T is homomorphic on the constructs already belonging to L;
2. every added Actor Graph construct is translated by a fixed context determined by its immediate syntax rather than by analysis or rewriting of the surrounding program; and
3. T preserves Obs_{AG}.

The Actor Graph constructs are **macro-expressible** in L, written

$$L_{AG} \preceq_{\text{macro}} L,$$

when an AG-local embedding exists. This adapts Felleisen's criterion to the model language defined in this paper; it is not presented as a new general definition of programming-language expressiveness [33].

15.4 Formal anchor and the remaining separation problem

Theorem 1 provides a necessary information condition for any faithful embedding: the admitted model class cannot collapse the LinkID of a structural relation, the TransactionID of an occurrence, and the ActorID of its mediator into one another. Corollary 1 shows the corresponding loss under untyped path replacement. Definitions 7 and 9 add further obligations: account mutation must remain transaction-attributable, and selected outer account observations must agree with identity-preserving disclosure of the internal graph.

These results constrain every encoding, but they do **not** by themselves prove that Actor Graphs are not macro-expressible in a particular host language. A compiler, language extension, runtime, database, or instrumentation layer may supply the missing discriminators and invariants. A genuine Felleisen-style separation result must therefore name a concrete pair (L_{AG}, L), a class of admissible translations, and an observation equivalence.

Open Conjecture (Actor Graph macro-expressibility boundary). Let L be a module-local host language whose core signature has no native equivalents of the three identity sorts, transaction-sourced actor accounts, and identity-preserving graph disclosure. Under Obs_{AG}, the full Actor Graph extension L_{AG} is not macro-expressible in L. Any faithful realisation requires either a non-local program transformation or an execution substrate that extends the host language with equivalent global mechanisms.

The conjecture is deliberately weaker than a claim that ordinary languages cannot implement Actor Graphs. They can. It asks whether they can do so by local expansion alone while preserving the observations fixed by this paper.

15.5 Limit expressiveness and representational absorption

Felleisen's ordering is relative to a base language, a translation class, and an observation relation. Consequently, **maximal expressive power** is not meaningful as an unconditional claim over all possible languages. This paper uses a narrower notion: maximality relative to the operational observation signature Obs_{AG} and to the class $\mathcal{S}_{op}$ of systems whose relevant observables are actors, relations, recognised occurrences, attributable state, causal evidence, and recursively disclosed processes.

For a conventional program-system $P \in \mathcal{S}_{op}$, consider a modelling map

$$\mathcal{M} : \mathcal{S}_{op} \longrightarrow \mathfrak{AG}_{\Sigma}$$

with the intended correspondence

$$\begin{aligned}
\text{module, object, service, person, or device} &\longmapsto \text{actor},\\
\text{call, channel, protocol, contract, or dependency} &\longmapsto \text{link plus edge actor},\\
\text{message, invocation, operation, or run} &\longmapsto \text{Transaction},\\
\text{persistent state, balance, limit, or obligation} &\longmapsto \text{Accounts},\\
\text{log, trace, observation, or evidence} &\longmapsto \text{Events},\\
\text{algorithm or executable code} &\longmapsto \text{LocalProcess}.
\end{aligned}$$

The Actor Graph signature is **representationally closed** for such a model when each mapped component remains inside the same signature under further disclosure: actors disclose into Actor Graphs, edge actors disclose into Actor Graphs, and residual algorithms remain `LocalProcess` values rather than forcing an exit to a second architectural representation.

Position and open conjecture (relative limit expressiveness). Relative to Obs_{AG} and $\mathcal{S}_{op}$, Actor Graph is a candidate example of **limit**, or **maximal native**, expressiveness: a conventional software system can be represented as an Actor Graph while preserving its local program text as actor processes, and each additional operational distinction required for faithful execution is represented by another construct of the same Actor Graph signature. Symbolically,

$$\mathsf{ClassicalProgramming} \preceq_{\text{model}} \mathsf{ActorGraphModeling}, \qquad P \longmapsto G_P[\text{LocalProcess} \leftarrow \text{code}(P)].$$

The relation $\preceq_{\text{model}}$ denotes representational containment, not greater computability and not a proved universal embedding theorem. Classical programming becomes a **special case inside** Actor Graph modelling: code supplies local transformations; the Actor Graph supplies the persistent identities, relations, transactions, accounts, events, and disclosure structure that make those transformations part of an operating system.

This is the precise sense of **absorption** used here. Actor Graphs do not erase lambda calculus or conventional programming languages. They model them, place them inside actor processes, and absorb the architectural functions that conventional programs otherwise reconstruct around their local code. The limiting thesis is that no separate code-centric representation is required as the authoritative model once both the program and the operational world in which it acts are represented by one recursively closed Actor Graph. This develops the earlier engineering statement that Actor Graphs model software itself, not only the domain around it [40].

The thesis remains conditional. A richer observation universe may require extending Σ; a concrete relative-maximality result requires a formal source language, a formal Actor Graph language, and a proof that $\mathcal{M}$ preserves the declared observations. The present paper identifies the candidate closure mechanism and states the programme; it does not promote the position to a theorem.

15.6 Where code-primary systems reconstruct the missing invariants

In code-primary enterprise systems, equivalents of the three discriminators commonly appear as database keys, correlation identifiers, idempotency keys, audit rows, queue headers, tracing spans, ledger entries, retry state, and compensation records. Their presence is consistent with Theorem 1: a faithful system must retain the distinctions somehow. The engineering problem is that the distinctions are often reconstructed separately by each service, channel, and storage layer rather than inherited from one native model signature.

Aspect-oriented programming identified transactions, tracing, security, and related concerns as cross-cutting because their implementation does not localise cleanly in the dominant functional decomposition [34]. Weaving can enforce such concerns across a program, but that observation reinforces the expressiveness distinction rather than settling it: weaving is compiler- or runtime-level transformation, not an ordinary local library expansion. Actor Graphs place the identity and accountability contract in the model signature itself.

Model-Driven Architecture likewise establishes models and transformations as first-class software-engineering artefacts [36]. Actor Graphs do not claim model primacy or code generation as new. Their narrower claim is that the executable model natively carries the specific identity, transaction, account, mediator, and disclosure invariants formalised here. MDA can encode or generate an Actor Graph implementation, but its general transformation framework does not itself impose this signature.

Dimension	**Code-primary host without AG primitives**	**Native Actor Graph signature**
Computability	General local computation	General local computation through `LocalProcess`
Persistent relation	Application/framework convention	`link_id` is primitive
Recognised occurrence	Correlation or audit convention	`transaction_id` is primitive
Reusable mediator	Object/service convention	edge actor with stable `actor_id`
Account mutation	Database/application discipline	transaction-sourced actor accounts
Cross-scale observation	Introspection and integration conventions	identity-preserving finite disclosure
Enforcement location	Libraries plus compiler/runtime/ program-wide coordination	invariant of the model class and execution specialisation

15.7 Model-primary construction

The programmatic implication is a reversal of source-of-truth priority. The executable Actor Graph model may become the authoritative representation of topology, identity, transactions, accounts, events, and disclosure, while conventional code remains inside `LocalProcess` as a replaceable implementation of domain-specific computation. Code is not eliminated; architectural code for glue, orchestration, identity propagation, audit, bookkeeping, and reconciliation is reduced to the extent that those functions are already carried by the model and runtime. This position is developed separately in *Beyond Programming Languages* [35]. In that architecture, classical programming is absorbed in the strict representational sense established above: its algorithms persist, but its program text ceases to be the sole or primary carrier of system identity, topology, state, history, and accountability.

The economic consequence is not a theorem. It is a falsifiable programme. Comparative implementations can measure, for example:

- the number of independently implemented identity, correlation, idempotency, audit, ledger, retry, and reconciliation mechanisms;

- the number of modules and stores that must change when one accountable relation changes;
- the amount of duplicated topology and policy across code, documentation, monitoring, and compliance artefacts;
- the ability to reconstruct one interaction from relation identity, transaction identity, mediator identity, postings, and trace without ad hoc joins; and
- the cost of migrating a code-primary system to a model-primary Actor Graph representation.

A reduction in those quantities would support the representation-economics hypothesis. Failure to reduce them would falsify or sharply limit it.

16 Contributions and claimed scope

The paper's novelty is one composition—**triple-identity accountable mediation**—supported by the following explicit contributions.

The programming-language expressiveness position is deliberately excluded from the proved contribution list. It remains an open programme anchored in Theorem 1, Corollary 1, and Definitions 7 and 9. By contrast, the categorical results below are proved, but their scope is explicitly restricted to resolved identity-incidence-accountability skeletons, fixed typing slices, and raw finite-disclosure maps.

1. **Named mathematical object.** Actor Graph is defined as a mathematical object with pluggable execution semantics rather than as a descriptive phrase.
2. **Identity-faithful representation and a non-collapse theorem.** Definition 6 formalises faithfulness by recoverability of the sorted identity signature; Theorem 1 proves that `link_id`, `transaction_id`, and `edge_actor_id` cannot be keyed solely by one another without collapsing admitted models.
3. **Universal actor roles.** Source, target, edge, external, and graph roles are occupied by actors from one identity universe and may overlap.
4. **Uniform accountable mediator contract.** Identity, accounts, transactions, events, and process apply uniformly to actors in every role, including the edge role.
5. **Transaction-sourced accounts.** Account mutation is attributable to a canonical committed transaction; event evidence remains distinct from transaction identity.
6. **Typed effect and balance algebra.** Account types have deterministic effect actions; commuting effects are order-independent; conserved quantitative types admit group-of-differences algebra and explicit boundary-relative balance.
7. **Account-coherent disclosure.** A graph actor may declare a partial aggregation under which outer account observations agree with a selected disclosure of its internal graph.
8. **Faithful typed reification.** A reversible typed incidence encoding is defined, together with a precise statement of what simpler reification loses.
9. **Canonical specification denotation.** A rooted registry-backed specification is constructed as an explicit F_Σ-coalgebra, and the unique final morphism supplies its coinductive denotation.
10. **Explicit coinductive functor and results.** A finitary polynomial signature functor includes accountable actors, canonical transaction records, links, and potential topology; final-coalgebra existence, finite observation, and full finite-observation adequacy are established.
11. **Functorial finite disclosure.** Theorem 4 promotes every raw depth-bounded disclosure to a functor into a slice category and proves strict compatibility with the final-sequence truncations.
12. **Categorical accountability skeleton.** Definitions 17-19 embed resolved ID-normal actor/link/transaction/account/posting structures into an Actor-Graph-specific schema, while Theorem 5

identifies the ambient schema-instance category with a presheaf category, including pointwise limits and colimits, sortwise monomorphisms, typed slices, and adhesive structural gluing.

13. **Categorical reconstruction obstruction.** Theorem 6 strengthens non-collapse: any representation functor that identifies one of the admitted witness pairs has no functorial left inverse on the full skeleton class.
14. **Structural fractality as a result.** The property is defined independently and proved for the Actor Graph class by final-coalgebra closure.
15. **Identity-, link-, occurrence-, and posting-preserving matrix family.** Primary adjacency stores edge-actor identities; incidence preserves links; transaction matrices preserve occurrences; posting matrices preserve account effects; classical matrices remain available as projections.
16. **Potential/realised distinction.** Potential topology specifies admissible source/mediator/target triples, including the mediator.
17. **Transactional specialisation boundary.** ATG is positioned as a strict closure of an already account- and transaction-bearing AG, adding transaction primacy, stronger invariants, and `(result, trace, ledger)` semantics.
18. **Explicit nearest-neighbour demarcation.** RDF 1.2, ORM, UML/SysML association classes, TypeDB, attributed graph transformation, Reo, recursive relation formalisms, DEMO/Enterprise Ontology, REA/OeBTO, double-entry algebra, presheaf graph categories, and adhesive categories are treated as genuine adjacent mechanisms; the claim is limited to their stated conjunction in the Actor Graph signature.

The paper does not claim isolated priority for any ingredient or a new general-purpose categorical construction. It proposes the name **Actor Graph** for the conjunction of actor-valued edges, separate relation/occurrence/mediator identities, a uniformly accountable and process-bearing mediator, identity-preserving coinductive disclosure, and an Actor-Graph-specific categorical accountability skeleton with native topology, occurrence, and posting attribution.

17 Limitations and open problems

17.1 Execution and commit semantics

The structural and coalgebraic object is fixed, and the account/transaction commit boundary is specified abstractly. Concrete distributed isolation, consensus, retry, idempotency, conflict resolution, fraud controls, and compensation protocols remain execution-specialisation concerns.

17.2 Spatial disclosure versus temporal execution

The coalgebra $\gamma : X \to F_\Sigma(X)$ gives a semantics of recursive **disclosure**: its recursive positions are graph-valued actor processes, while account state, events, transaction-history prefixes, and transaction data are constant coordinates of one observed configuration. It is therefore a semantics of spatial and organisational depth, not yet a temporal transition system. The commit step $c \xrightarrow{\tau} c'$ and side condition S2 remain external to the F_Σ-coalgebraic denotation. A complete operational theory must introduce a temporal behaviour functor and prove compatibility among commit, disclosure, and open composition. Bialgebraic operational semantics and abstract GSOS provide a natural framework for that programme [55], but no distributive law or congruence theorem is claimed in the present paper.

17.3 Behavioural quotients

The adequacy theorem uses exact identity-preserving observations unless a domain declares quotient relations for accounts, transaction data, events, or local processes. Constructing useful

quotients without erasing legally or economically relevant distinctions is a major domain problem.

17.4 Process-aware path algebra

Semiring matrices compose path labels, but they do not yet compose actor processes, account effects, transaction preconditions, authority, enablement, compensation, or recursive disclosure. A process- and ledger-aware algebra remains open.

17.5 Infinite histories and finite analysis

The functor stores finite transaction-history prefixes at each finite configuration. Long-running systems generate unbounded histories over time. Incremental, windowed, and cryptographically committed representations must preserve transaction identity and auditability without requiring whole-history materialisation.

17.6 Identity governance

The mathematics assumes stable ActorID, LinkID, and TransactionID domains. Real systems need issuance, namespace federation, collision prevention, versioning, provenance, revocation, delegation, privacy, and legal recognition.

17.7 Reuse of edge actors

One edge actor may perform many links and many transactions. The formalism preserves that fact, but practical governance must decide how state, limits, fees, failure, accountability, and transaction histories are partitioned or shared across occurrences.

17.8 Fractal invariants

Structural fractality is proved at the signature level. Domain-specific invariants—account conservation, transaction admissibility, security policy, authority, service levels, or jurisdictional constraints—require separate compositional proofs across disclosure boundaries.

17.9 Scale and storage

Vector-valued topology, transaction matrices, account postings, and recursive observations can be large. Sparse storage, indexing, streaming, incremental matrix maintenance, and partial disclosure require engineering analysis.

17.10 Valuation, conversion, and non-quantitative accounts

The account algebra fixes typing, attribution, effect composition, balance for declared conserved types, and disclosure coherence. It does not choose exchange rates, valuation rules, a chart of accounts, legal consolidation standards, or the effect monoids appropriate to documents, statuses, authorities, and other non-quantitative domains. Multi-type conversions require an explicit conversion transaction and policy; conservation is then evaluated per declared type and boundary.

17.11 Programming-language separation

Definition 24 fixes a Felleisen-style criterion, but the paper proves no non-macro-expressibility theorem for a concrete host language. Such a theorem requires a formal syntax and semantics for

both the host and Actor Graph extension, a precisely delimited translation class, and an observation equivalence. Compiler plugins, aspect weavers, effect systems, databases, and runtimes may change the answer by enriching the host language.

17.12 Model-primary construction

The claim that a native Actor Graph representation reduces duplication, fragmentation, and redundancy is empirical. Evaluation requires comparative case studies with declared metrics, including counts of independent identity, idempotency, audit, reconciliation, and transaction-tracing mechanisms before and after migration. The formal non-collapse theorem predicts which distinctions must exist; it does not determine their engineering cost.

17.13 Relative maximality and software absorption

The statement that Actor Graph is a candidate limit-expressive representation is relative to Obs_{AG} and $\mathcal{S}_{op}$. The paper does not prove that every programming formalism has a faithful observation-preserving embedding into Actor Graphs, nor that no richer native signature exists. Establishing relative maximality requires a formal modelling map for a specified source language or runtime and a proof of observation preservation. The term **absorption** means representational containment of classical code as `LocalProcess`, not semantic elimination of programming languages or proof of absolute maximal expressive power.

17.14 Full enriched composition and categorical completion

Theorem 5 establishes an ambient presheaf category for resolved identity-incidence-accountability skeletons, not a topos characterisation of the full enriched and coinductive Actor Graph class. Event order, transaction-history consistency, registry coherence, graph-valued processes, potential-link policy, legal authority, and balance/aggregation obligations remain additional structure or admissibility predicates.

Open Actor Graphs should be composable through declared actor and accounting interfaces. A candidate presentation is a structured or decorated cospan [46, 47]:

$$L(I) \longrightarrow G \longleftarrow L(O).$$

Theorem 5 and Corollary 2 supply the finite structural pushouts needed by this programme. They do not yet prove that an enriched pushout preserves every Actor Graph invariant. A complete theorem must specify admissible interfaces, distinguish strict sharing from coordinated renaming, reconcile actor state and histories, preserve transaction binding, and internalise matched boundary postings.

For a declared conserved type κ, the target result is a boundary-conservation theorem for an explicitly defined enriched composition. It must distinguish two statements: structural gluing may preserve global balance while retaining the shared interface as internal accountable state, whereas elimination of the interface requires an additional zero-seam condition on its opposed boundary effects. Concretely, for every conserved type κ and every identified seam account coordinate q, the two oriented boundary effects must satisfy

$$e^{\kappa}_{1,J}(q) \oplus_{\kappa} e^{\kappa}_{2,J}(q) = 0_{\kappa};$$

equivalently, the seam-effect vector is zero. Aggregate cancellation alone does not justify interface elimination when non-zero effects are redistributed among seam accounts. Neither conservation nor this stronger cut condition follows from presheaf adhesivity alone.

Theorem 4 completes the functoriality of raw finite disclosure. A categorical characterisation of identity-preserving bisimulation by spans of coalgebra morphisms or open maps remains open [45]. Account-coherent aggregation remains only partially categorical. A further treatment should model

disclosure views as indexed or partial structure and express account aggregation as a natural or lax-natural transformation between inner and outer observation functors. It remains premature to call disclosure a comonad until an endofunctor, counit, comultiplication, and their laws are specified on a category carrying the required enriched structure.

Dependent-polynomial or indexed signatures may internalise more well-formedness data [48]. They will not automatically absorb global namespace governance, consistency of external registries, distributed uniqueness, legal authority, privacy, or cross-jurisdiction recognition. These are not merely local presheaf coordinates.

The companion manuscript **The Category of Open Actor Graphs: Accountable Composition, Boundary Conservation, and the Zero-Seam Cut Rule** executes this finite resolved composition programme. The remaining larger burden is dynamic: introduce temporal commit behaviour and prove its compatibility with recursive disclosure and open composition rather than treating execution as an external specialisation.

17.15 Empirical validation

The present paper is definitional and formal. Comparative case studies are needed to measure whether the native Actor Graph representation improves modelling fidelity, migration, simulation, auditability, or operational change relative to alternative formalisms.

18 Conclusion

An Actor Graph is a directed multigraph in which vertices are actors and every realised edge role is performed by an actor from the same universal identity-bearing type, while the structural link retains an identity distinct from its mediator; every actor carries accounts, transactions, events, and process. A link describes persistent topology. A transaction describes one identified occurrence through that topology. Account effects, transaction histories, and event evidence remain attributable to the actors that produced and mediated the transition.

Identity faithfulness is stated as recoverability of the sorted link, transaction, and mediator signature. The non-collapse theorem then shows that no exact representation of the admitted model class can key one of those three identity sorts solely by another. A registry-backed specification has an explicit coalgebraic semantics: resolving one graph handle defines an F_Σ-coalgebra step, and the unique final morphism maps the rooted specification to its coinductive denotation.

The recursive definition is coinductive. A finitary polynomial signature functor explicitly contains actor accounts, transaction histories, canonical transaction records, links, potential topology, and graph-valued process positions. The final coalgebra supplies the recursive carrier; finite observations are order-independent; and, for locally finite specifications, equality of all canonical finite observations is adequate for identity-preserving bisimulation. Direct and indirect cyclic references are legitimate and need not be unfolded eagerly.

At the resolved accountability layer, the identity-incidence-accountability data have an ID-normal form with actor, link, transaction, account-coordinate, and posting-occurrence sorts. Their ambient schema category is the presheaf category $[\mathbb{S}_{\mathrm{AG}}, \mathbf{Set}]$. Structural limits and colimits are therefore pointwise, monomorphisms are sortwise injective, and namespace-safe pushouts provide adhesive structural gluing. This result is deliberately not extended by fiat to the full policy-constrained coalgebraic class. The categorical no-reconstruction theorem strengthens non-collapse by excluding any functorial decoder after a representation has identified one of the link/transaction/mediator witness pairs. Independently, the raw finite disclosures form a strict functorial truncation tower.

Structural fractality is not an additional mystique or geometric claim. It is the proved closure of the Actor Graph signature under disclosure: every disclosed graph again contains actors, actor-valued edges, links, accounts, transactions, events, and further graph-valued processes of the same type.

The matrix layer reflects the same hierarchy. The primary adjacency matrix preserves edge-

actor identities. Incidence matrices preserve links. Transaction matrices preserve occurrences. Account-posting matrices preserve ledger effects. Numeric projections recover classical graph analysis but cannot reconstruct the acting relations or the transactions that changed their accounts.

On the programming-language axis, the result is not greater computability. Lambda-calculus terms, functions, and conventional programs remain available inside `LocalProcess`. The sharper question is representational expressiveness: the Actor Graph signature carries link, occurrence, and mediator identities, transaction-sourced accounts, causal events, and recursive disclosure natively; a code-primary language must either express them by local expansion or reconstruct them through compiler/runtime and program-wide mechanisms. Definition 24 states the comparison criterion. Relative to the declared operational observations, Actor Graph is proposed as a candidate limit-expressive model in which classical programming is contained and absorbed as the local process layer. The separation, relative-maximality, and representation-economics hypotheses remain open.

Active Transaction Graphs then impose the stricter discipline that every recognised interaction is transaction-primary and execution is judged by result, trace, and ledger. The result is a publication-level foundation for **Actor Graph** as a precise term: a coinductive, structurally fractal, account- and transaction-bearing graph whose relations are themselves actors and whose identities remain stable across role, occurrence, scale, and recursive disclosure.

Data and source availability

The patent publication, related papers, canonical PDFs, readable Markdown sources, and citation metadata are linked below. The curated corpus is maintained in Corezoid Research. The one-page *Simulator.Company Basics* note cited for the historical lambda notation is included as supplementary material in the publication package. This paper contains no empirical data set; its formal definitions, propositions, and worked example are fully stated in the manuscript.

Disclosure of interest

The author is the founder of Corezoid Inc. and the inventor named in the *Actor Graph Engine* patent application. The patent was filed under the legal name **Oleksandr Vityaz**; the author publishes this manuscript as **Alexander Vityaz**. Corezoid and Simulator.Company are implementations developed by the author's company. This manuscript defines and analyses the Actor Graph formalism; it is not an independent product evaluation.

Selected related publications by the author

The following publications develop adjacent parts of the same research programme.

1. **Actor Graph Engine** — architectural disclosure of actors as graph nodes and edge actors, account trees, transaction events and double-entry changes, recursive graph actors, layers, and structural self-similarity.
   Patent record
2. **Active Transaction Graphs: A Formal Framework for Transactional Interactive Systems** — execution specialisation of Actor Graphs, including transaction primacy, first-class edge mediation, recursive actors, and (result, trace, ledger) semantics.
   doi:10.5281/zenodo.20747873 · ResearchGate
3. **On the Nature of the Regulator: A Symposium on Frameworks and Actor Graphs** — positions the Actor Graph as a macro-framework for regulation and AI as a local framework within it.

doi:10.13140/RG.2.2.30218.02244 · ResearchGate

4. **Company Brain: The Architecture of General Company Intelligence** — applies Actor Graphs to the computable geometric model and digital twin of an organisation.
   doi:10.13140/RG.2.2.28274.88007 · ResearchGate
5. **A Phase Model of Enterprise Evolution: From Fragmentation to the Autonomous Enterprise** — places Actor Graphs in the transition from a digital core to a digital twin and autonomous enterprise.
   doi:10.13140/RG.2.2.24883.39207 · ResearchGate
6. **The Computable Boundary of the Firm: Information Conditions for Viability and the Transactional Architecture of the Digital Twin** — develops actor membership, boundary observability, permeability, and the Executable Boundary Actor.
   doi:10.5281/zenodo.20745927 · ResearchGate
7. **Management Debt—Part I: Concept, Metrics, and Principles for Attributing Materialised Debts to Actor Accounts** — develops attributable actor accounts and ledger consequences for managerial omissions.
   doi:10.5281/zenodo.21069692 · ResearchGate
8. **Beyond Programming Languages** — argues that verified Actor Graphs can become the source of operational truth while code becomes a regenerable process implementation.
   doi:10.5281/zenodo.21458098 · ResearchGate
9. **The Compact Company: An Actor-Graph Theory of the Firm in the AI Era** — applies a typed temporal Actor Graph to organisational closure, firm boundaries, continuity, and the minimum human core.
   doi:10.5281/zenodo.21774758 · ResearchGate
10. **Ontology of Transition: Causal Order, External Time, and the Thermodynamics of Physical Clock Records** — develops the event order, physical records, and operational time used by dynamic Actor Graph models.
    doi:10.5281/zenodo.21380580 · ResearchGate
11. **On the Necessity of Noise Suppression for Minimal Good Regulators: Factorization Theorems and a Closure Conjecture** — provides the cybernetic factorisation result used by several Actor Graph applications.
    doi:10.13140/RG.2.2.33143.07843 · ResearchGate

Research corpus: Corezoid Research
ResearchGate profile: Alexander Vityaz

References

[1] Vityaz, O. *Actor Graph Engine*. U.S. Patent Application Publication US20240378338A1, filed 8 May 2023, published 14 November 2024. In particular §§0054-0060, 0067-0079, 0109-0119, 0121-0140, 0161-0186; Figs. 2-25; claims 4, 12, and 19. Patent record

[2] Vityaz, A. *Active Transaction Graphs: A Formal Framework for Transactional Interactive Systems*. Zenodo, 2026. doi:10.5281/zenodo.20747873

[3] Diestel, R. *Graph Theory*. 5th ed. Springer, 2017. doi:10.1007/978-3-662-53622-3

[4] Harary, F. *Graph Theory*. Addison-Wesley, 1969.

[5] Hewitt, C., Bishop, P., and Steiger, R. "A Universal Modular ACTOR Formalism for Artificial Intelligence." *Proceedings of IJCAI*, 1973, pp. 235-245.

[6] Agha, G. *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press, 1986.

[7] Petri, C. A. *Kommunikation mit Automaten*. Doctoral dissertation, Technische Hochschule Darmstadt, 1962.

[8] Milner, R. “Axioms for Bigraphical Structure.” *Mathematical Structures in Computer Science* 15(6), 2005, pp. 1005-1032. doi:10.1017/S0960129505004809

[9] Cardelli, L., and Gordon, A. D. “Mobile Ambients.” *Theoretical Computer Science* 240(1), 2000, pp. 177-213. doi:10.1016/S0304-3975(99)00231-5

[10] Harel, D. “Statecharts: A Visual Formalism for Complex Systems.” *Science of Computer Programming* 8(3), 1987, pp. 231-274. doi:10.1016/0167-6423(87)90035-9

[11] Zeigler, B. P., Praehofer, H., and Kim, T. G. *Theory of Modeling and Simulation*. 2nd ed. Academic Press, 2000.

[12] Davis, J. et al. *Ptolemy II: Heterogeneous Concurrent Modeling and Design in Java*. UCB/ERL M99/40, University of California, Berkeley, 1999.

[13] van der Aalst, W. M. P. *Process Mining: Data Science in Action*. 2nd ed. Springer, 2016. doi:10.1007/978-3-662-49851-4

[14] Angles, R., Arenas, M., Barceló, P., Hogan, A., Reutter, J., and Vrgoc, D. “Foundations of Modern Query Languages for Graph Databases.” *ACM Computing Surveys* 50(5), 2017, Article 68. doi:10.1145/3104031

[15] W3C RDF & SPARQL Working Group. *RDF 1.2 Concepts and Abstract Data Model*. W3C Candidate Recommendation Snapshot, 7 April 2026. Dated snapshot

[16] TypeDB. *Entities, Relations, Attributes* and *Graph Database Use Case*. Official documentation, accessed 11 August 2026. Core concepts · Relations playing roles

[17] Basu, A., and Blanning, R. W. *Metagraphs and Their Applications*. Springer, 2007. doi:10.1007/978-0-387-37234-1

[18] Joslyn, C., and Nowak, K. “Ubergraphs: A Definition of a Recursive Hypergraph Structure.” arXiv:1704.05547, 2017. doi:10.48550/arXiv.1704.05547

[19] Harel, D. “On Visual Formalisms.” *Communications of the ACM* 31(5), 1988, pp. 514-530. doi:10.1145/42411.42414

[20] Drewes, F., Hoffmann, B., and Plump, D. “Hierarchical Graph Transformation.” *Journal of Computer and System Sciences* 64(2), 2002, pp. 249-283. doi:10.1006/jcss.2001.1790

[21] Droste, M., Kuich, W., and Vogler, H., eds. *Handbook of Weighted Automata*. Springer, 2009. doi:10.1007/978-3-642-01492-5

[22] Rutten, J. J. M. M. “Universal Coalgebra: A Theory of Systems.” *Theoretical Computer Science* 249(1), 2000, pp. 3-80. doi:10.1016/S0304-3975(00)00056-6

[23] Kafura, D., Washabaugh, D., and Nelson, J. “Garbage Collection of Actors.” *OOPSLA/ECOOP 1990*, pp. 126-134. doi:10.1145/97945.97961

[24] Lukose, D., and Mineau, G. W. “A Comparative Study of Dynamic Conceptual Graphs.” *Proceedings of KAW-98*, Banff, 1998. Source, accessed 17 August 2026

[25] Yadati, N. et al. “Neural Message Passing for Multi-Relational Ordered and Recursive Hypergraphs.” *Advances in Neural Information Processing Systems* 33, 2020.

[26] Worrell, J. “On the Final Sequence of a Finitary Set Functor.” *Theoretical Computer Science* 338(1-3), 2005, pp. 184-199. doi:10.1016/j.tcs.2004.12.009

[27] McCarthy, W. E. “The REA Accounting Model: A Generalized Framework for Accounting Systems in a Shared Data Environment.” *The Accounting Review* 57(3), 1982, pp. 554-578. Publisher page

[28] Geerts, G. L., and McCarthy, W. E. “Policy-Level Specifications in REA Enterprise Information Systems.” *Journal of Information Systems* 20(2), 2006, pp. 37-63. doi:10.2308/jis.2006.20.2.37

[29] ISO/IEC 15944-4:2015. *Information technology — Business Operational View — Part 4: Business transaction scenarios — Accounting and economic ontology*. ISO, 2015. Standard record

[30] Ellerman, D. P. “The Mathematics of Double Entry Bookkeeping.” *Mathematics Magazine* 58(4), 1985, pp. 226-233. doi:10.2307/2689520

[31] Ellerman, D. P. “On Double-Entry Bookkeeping: The Mathematical Treatment.” *Accounting Education* 23(5), 2014, pp. 483-501. doi:10.1080/09639284.2014.949803

[32] Hruby, P. *Model-Driven Design Using Business Patterns*. Springer, 2006. doi:10.1007/3-540-30327-2

[33] Felleisen, M. “On the Expressive Power of Programming Languages.” *Science of Computer Programming* 17(1-3), 1991, pp. 35-75. doi:10.1016/0167-6423(91)90036-W

[34] Kiczales, G., Lamping, J., Mendhekar, A., Maeda, C., Lopes, C. V., Loingtier, J.-M., and Irwin, J. “Aspect-Oriented Programming.” In Akşit, M., and Matsuoka, S., eds., *ECOOP 1997 — Object-Oriented Programming*, LNCS 1241, Springer, 1997, pp. 220-242. doi:10.1007/BFb0053381

[35] Vityaz, A. *Beyond Programming Languages*. Zenodo, 2026. doi:10.5281/zenodo.21458098

[36] Object Management Group. *MDA Guide, Revision 2.0*. OMG Document ormsc/2014-06-01, 2014. Official guide

[37] Church, A. “An Unsolvable Problem of Elementary Number Theory.” *American Journal of Mathematics* 58(2), 1936, pp. 345-363. doi:10.2307/2371045

[38] Barendregt, H. P. *The Lambda Calculus: Its Syntax and Semantics*. Revised ed. North-Holland, 1984.

[39] Landin, P. J. “A Correspondence Between ALGOL 60 and Church’s Lambda-notation: Part I.” *Communications of the ACM* 8(2), 1965, pp. 89-101. doi:10.1145/363744.363749

[40] Corezoid Inc. *Simulator.Company Basics*. Technical note, 2025.

[41] Arbab, F. “Reo: A Channel-Based Coordination Model for Component Composition.” *Mathematical Structures in Computer Science* 14(3), 2004, pp. 329-366. doi:10.1017/S0960129504004153

[42] Baier, C., Sirjani, M., Arbab, F., and Rutten, J. J. M. M. “Modeling Component Connectors in Reo by Constraint Automata.” *Science of Computer Programming* 61(2), 2006, pp. 75-113. doi:10.1016/j.scico.2005.10.008

[43] Dietz, J. L. G. *Enterprise Ontology: Theory and Methodology*. Springer, 2006. doi:10.1007/3-540-33149-2

[44] Halpin, T. “Objectification of Relationships.” In Siau, K., ed., *Advanced Topics in Database Research*, vol. 5. Idea Group, 2006, pp. 106-123. Book record

[45] Joyal, A., Nielsen, M., and Winskel, G. “Bisimulation from Open Maps.” *Information and Computation* 127(2), 1996, pp. 164-185. doi:10.1006/inco.1996.0057

[46] Fong, B. “Decorated Cospans.” *Theory and Applications of Categories* 30(33), 2015, pp. 1096-1120. Journal PDF

[47] Baez, J. C., and Courser, K. “Structured Cospans.” *Theory and Applications of Categories* 35(48), 2020, pp. 1771-1822. Journal PDF

[48] Gambino, N., and Kock, J. “Polynomial Functors and Polynomial Monads.” *Mathematical Proceedings of the Cambridge Philosophical Society* 154(1), 2013, pp. 153-192. doi:10.1017/S0305004112000394

[49] Mac Lane, S., and Moerdijk, I. *Sheaves in Geometry and Logic: A First Introduction to Topos Theory*. Universitext. Springer, 1992. doi:10.1007/978-1-4612-0927-0

[50] Lack, S., and Sobociński, P. “Adhesive and Quasiadhesive Categories.” *RAIRO—Theoretical Informatics and Applications* 39(3), 2005, pp. 511-545. doi:10.1051/ita:2005028

[51] Lack, S., and Sobociński, P. “Toposes Are Adhesive.” In *Graph Transformations: ICGT 2006*, LNCS 4178, Springer, 2006, pp. 184-198. doi:10.1007/11841883_14

[52] Object Management Group. *OMG Unified Modeling Language (OMG UML), Version 2.5.1*. OMG Document formal/17-12-05, 2017. Normative specification

[53] Object Management Group. *OMG Systems Modeling Language (OMG SysML), Version 1.7*. OMG Document formal/24-01-07, 2024. See the AssociationBlock and ConnectorProperty clauses. Specification record

[54] Ehrig, H., Ehrig, K., Prange, U., and Taentzer, G. *Fundamentals of Algebraic Graph Transformation*. Monographs in Theoretical Computer Science. Springer, 2006. doi:10.1007/3-540-31188-2

[55] Turi, D., and Plotkin, G. D. “Towards a Mathematical Operational Semantics.” *Proceedings of the 12th Annual IEEE Symposium on Logic in Computer Science*, 1997, pp. 280-291. Author PDF

[56] Spivak, D. I. “Functorial Data Migration.” Information and Computation 217, 2012, pp. 31-51. doi:10.1016/j.ic.2012.05.001

[57] Lebo, T., Sahoo, S., McGuinness, D., et al. PROV-O: The PROV Ontology. W3C Recommendation, 30 April 2013. Official recommendation

[58] Berti, A. et al. “OCEL (Object-Centric Event Log) 2.0 Specification.” arXiv:2403.01975, 2024. doi:10.48550/arXiv.2403.01975

[59] Cariou, E., and Beugnard, A. “The Specification of UML Collaborations as Interaction Components.” In UML 2002 — The Unified Modeling Language, LNCS 2460, Springer, 2002, pp. 352-367. doi:10.1007/3-540-45800-X_28

[60] Bierman, G., and Wren, A. “First-Class Relationships in an Object-Oriented Language.” In ECOOP 2005 — Object-Oriented Programming, LNCS 3586, Springer, 2005, pp. 262-286. doi:10.1007/11531142_12